Namal University, Mianwali

Department of Electrical Engineering

EE-474L – Computer Architecture (Lab)

Design Project Report

Design and Implementation of a 32-bit, 5-Stage Pipelined RISC-V Processor (RV32I)

Name	Hassan Ali Ahmad
Reg No.	NUM-BSEE-2023-03
Submission Date	17 June 2026

Course Instructor:

Dr. Tassadaq Hussain

Lab Instructor:

Engr. Rizwana Khan

Contents

Table of Figures:

Abstract

This project presents the design and implementation of a 32-bit RISC-V processor with a five-stage pipeline following the RV32I instruction set. The processor performs instruction fetch, decode, execute, memory access, and write back in separate stages to improve performance. It handles data and control hazards using techniques such as forwarding and pipeline stalling, ensuring that arithmetic, logical, memory, and branch instructions execute correctly. The processor was verified using C programs compiled for RISCV, and simulation results confirmed correct operation and proper pipeline behavior. Performance analysis, including average cycles per instruction, shows that the processor works efficiently.

1. Introduction

So we going to design the 5 stage pipelined RV32I processor with data and control hazards. Let's dive into what is meant by this?

First we will understand what is RISCV processor:

1.1. RISCV Processor

A RISC-V processor is a type of computer processor that follows the RISC-V instruction set architecture (ISA)**.** RISC-V is an open standard, which means it is free to use, modify, and implement without any licensing restrictions. The architecture is based on the RISC (Reduced Instruction Set Computing) principles, which aim to simplify the instruction set, reduce complexity, and improve performance.

Now let's understand what does the RV32I mean:

1.2. RV32I

RV32I is the base integer instruction set of the RISC-V architecture. The "RV" stands for RISCV, "32" indicates a 32-bit processor, and "I" stands for integer, meaning it supports basic integer operations like addition, subtraction, and logical operations. RV32I provides all the essential instructions needed for general-purpose computing, including arithmetic, logical, memory load and store, branches, jumps, and system-level instructions. It forms the foundation for more advanced extensions, but even on its own, RV32I is enough to run real programs compiled from high-level languages like C.

RV32I supports a total of 47 base instructions. These instructions cover all the essential operations required for general-purpose computing and can be grouped as follows:

1.2.1. R-Type Instructions

- **ADD**: Adds two registers and stores the result in a register.
- **SUB**: Subtracts one register from another and stores the result.

- **SLL**: Shifts the value in a register left by a number of bits.
- **SLT**: Sets the destination register to 1 if one register is less than another (signed).
- **SLTU**: Sets the destination register to 1 if one register is less than another (unsigned).
- **XOR**: Performs bitwise XOR between two registers.
- **SRL**: Shifts the value in a register right logically (fills with zeros).
- **SRA**: Shifts the value in a register right arithmetically (preserves sign).
- **OR**: Performs bitwise OR between two registers.
- **AND**: Performs bitwise AND between two registers.

1.2.2. I-Type Instructions

- **ADDI**: Adds a register and an immediate value.
- **SLLI**: Shifts a register value left by an immediate value.
- **SLTI**: Sets destination to 1 if a register is less than an immediate (signed).
- **SLTIU**: Sets destination to 1 if a register is less than an immediate (unsigned).
- **XORI**: Performs bitwise XOR with a register and an immediate.
- **SRLI**: Shifts a register right logically by an immediate.
- **SRAI**: Shifts a register right arithmetically by an immediate.
- **ORI**: Performs bitwise OR with a register and an immediate.
- **ANDI**: Performs bitwise AND with a register and an immediate.
- **JALR**: Jumps to an address and stores return address in a register.

1.2.3. Load Instructions

- **LB**: Loads a byte from memory and sign-extends it.
- **LH**: Loads a halfword (16 bits) from memory and sign-extends it.
- **LW**: Loads a word (32 bits) from memory.
- **LBU**: Loads a byte from memory and zero-extends it.
- **LHU**: Loads a halfword from memory and zero-extends it.

1.2.4. Store Instructions

- **SB**: Stores a byte from a register to memory.
- **SH**: Stores a halfword from a register to memory.
- **SW**: Stores a word from a register to memory.

1.2.5. Branch Instructions

- **BEQ**: Branches if two registers are equal.
- **BNE**: Branches if two registers are not equal.
- **BLT**: Branches if one register is less than another (signed).
- **BGE**: Branches if one register is greater than or equal to another (signed).
- **BLTU**: Branches if one register is less than another (unsigned).
- **BGEU**: Branches if one register is greater than or equal to another (unsigned).

1.2.6. Jump Instructions

- **JAL**: Jumps to an address and stores the return address in a register.

Here is the complete ISA of EV32I:

31 27	26 25	24 20	19 15	14 12	11 7	6 0	
funct7		rs2	rs1	funct3	rd	opcode	R-type
imm[11:0]			rs1	funct3	rd	opcode	I-type
imm[11:5]		rs2	rs1	funct3	imm[4:0]	opcode	S-type
imm[12|10:5]		rs2	rs1	funct3	imm[4:1|11]	opcode	B-type
imm[31:12]					rd	opcode	U-type
imm[20|10:1|11|19:12]					rd	opcode	J-type

RV32I Base Instruction Set

imm[31:12]					rd	0110111	LUI
imm[31:12]					rd	0010111	AUIPC
imm[20|10:1|11|19:12]					rd	1101111	JAL
imm[11:0]			rs1	000	rd	1100111	JALR
imm[12|10:5]		rs2	rs1	000	imm[4:1|11]	1100011	BEQ
imm[12|10:5]		rs2	rs1	001	imm[4:1|11]	1100011	BNE
imm[12|10:5]		rs2	rs1	100	imm[4:1|11]	1100011	BLT
imm[12|10:5]		rs2	rs1	101	imm[4:1|11]	1100011	BGE
imm[12|10:5]		rs2	rs1	110	imm[4:1|11]	1100011	BLTU
imm[12|10:5]		rs2	rs1	111	imm[4:1|11]	1100011	BGEU
imm[11:0]			rs1	000	rd	0000011	LB
imm[11:0]			rs1	001	rd	0000011	LH
imm[11:0]			rs1	010	rd	0000011	LW
imm[11:0]			rs1	100	rd	0000011	LBU
imm[11:0]			rs1	101	rd	0000011	LHU
imm[11:5]		rs2	rs1	000	imm[4:0]	0100011	SB
imm[11:5]		rs2	rs1	001	imm[4:0]	0100011	SH
imm[11:5]		rs2	rs1	010	imm[4:0]	0100011	SW
imm[11:0]			rs1	000	rd	0010011	ADDI
imm[11:0]			rs1	010	rd	0010011	SLTI
imm[11:0]			rs1	011	rd	0010011	SLTIU
imm[11:0]			rs1	100	rd	0010011	XORI
imm[11:0]			rs1	110	rd	0010011	ORI
imm[11:0]			rs1	111	rd	0010011	ANDI
0000000		shamt	rs1	001	rd	0010011	SLLI
0000000		shamt	rs1	101	rd	0010011	SRLI
0100000		shamt	rs1	101	rd	0010011	SRAI
0000000		rs2	rs1	000	rd	0110011	ADD
0100000		rs2	rs1	000	rd	0110011	SUB
0000000		rs2	rs1	001	rd	0110011	SLL
0000000		rs2	rs1	010	rd	0110011	SLT
0000000		rs2	rs1	011	rd	0110011	SLTU
0000000		rs2	rs1	100	rd	0110011	XOR
0000000		rs2	rs1	101	rd	0110011	SRL
0100000		rs2	rs1	101	rd	0110011	SRA
0000000		rs2	rs1	110	rd	0110011	OR
0000000		rs2	rs1	111	rd	0110011	AND
0000	pred	succ	00000	000	00000	0001111	FENCE
0000	0000	0000	00000	001	00000	0001111	FENCE.I
000000000000			00000	000	00000	1110011	ECALL
000000000001			00000	000	00000	1110011	EBREAK
csr			rs1	001	rd	1110011	CSRRW
csr			rs1	010	rd	1110011	CSRRS
csr			rs1	011	rd	1110011	CSRRC
csr			zimm	101	rd	1110011	CSRRWI
csr			zimm	110	rd	1110011	CSRRSI
csr			zimm	111	rd	1110011	CSRRCI

Figure 1: RV32I ISA

Now it's turn to understand the meaning of "5-stage":

1.3. 5 stage RISCV Processor

The term “5-stage” refers to the five separate steps that an instruction goes through in a pipelined processor. Instead of executing each instruction one at a time from start to finish, the processor divides instruction execution into five stages. Each stage performs a specific task, and multiple instructions can be processed simultaneously, each at a different stage. This increases the overall performance of the processor by allowing instructions to overlap in time.

The five stages are:

1.3.1. Instruction Fetch (IF)

In this step, the processor reads the instruction from instruction memory using the program counter. After reading the instruction, the program counter is updated so the next instruction can be accessed.

1.3.2. Instruction Decode (ID)

In this step, the processor understands what the instruction is asking to do. It reads the required values from the registers and prepares the control signals needed for execution. If the instruction is a branch, the target address is also calculated here.

1.3.3. Execute (EX)

In this step, the main operation of the instruction is performed. Arithmetic and logical calculations are done using the ALU. For load and store instructions, the memory address is calculated. For branch instructions, the condition is checked.

1.3.4. Memory Access (MEM)

This step is used only by instructions that need memory. Load instructions read data from memory, while store instructions write data to memory. Instructions that do not use memory simply move forward without any action.

1.3.5. Write Back (WB)

In the final step, the result of the instruction is written back into the destination register. After this step, the instruction is completely finished.
Now let’s dive into the meaning of pipelining:

1.4. Pipelining

Pipelining is a technique used in processors to improve performance by allowing different parts of multiple instructions to be processed at the same time. Instead of waiting for one instruction to completely finish before starting the next one, the processor breaks instruction execution into stages and works on several instructions together. Each instruction is at a different stage, but all stages operate at the same time.

This approach increases the number of instructions completed in a given amount of time. After the pipeline is filled, the processor can complete almost one instruction in every clock cycle. Pipelining does not make a single instruction faster, but it increases the overall speed of program execution.

However, pipelining can also create problems when instructions depend on each other or when the flow of execution changes. These problems are called hazards, and special techniques are needed to handle them so that the processor produces correct results.

1.5. Hazard Types

There are three types of hazards which are given below:

1.5.1. Data Hazards

A data hazard happens when an instruction needs a value that is produced by a previous instruction, but that value is not yet available.

Example:

```
ADD x5, x1, x2
SUB x6, x5, x3
```

In this example, the first instruction adds the values of registers x1 and x2 and stores the result in register x5. The second instruction immediately uses the value of x5 to perform subtraction. A data hazard occurs because the second instruction needs the updated value of x5 before it is fully written back.

Solutions:

This processor uses forwarding logic to handle most data hazards. When forwarding cannot resolve the dependency, pipeline stalling is applied to ensure correct execution.

1.5.2. Control Hazards

A control hazard occurs when the processor encounters a branch or jump instruction. The next instruction depends on the outcome of the branch, which is not known immediately.

Example:

```
BEQ x1, x2, label
ADD x3, x4, x5
label:
SUB x6, x7, x8
```

Here, the BEQ instruction checks whether x1 and x2 are equal. The processor does not immediately know whether the branch will be taken. Meanwhile, the next instruction (ADD) may already start executing. If the branch is taken, the ADD instruction should not be executed, which creates a control hazard.

Solutions:

Control hazards are handled by stalling the pipeline or flushing incorrect instructions.

- **Stalling:** The processor waits until the branch decision is known.
- **Flushing:** Instructions that were fetched incorrectly are removed from the pipeline.

1.5.3. Structural Hazards:

A structural hazard happens when two instructions try to use the same hardware resource at the same time.

Example:

```
sw x5, 4(x1)
lw x6, 6(x1)
```

In this example, the LW instruction needs access to data memory to read a value, while the SW instruction also needs access to data memory to write a value. If the processor uses a single shared memory for both instruction fetch and data access, a conflict can occur when memory is required by more than one operation at the same time.

Solutions:

Structural hazards are completely avoided by design by using separate memories for instructions and data or using stalling techniques.

2. System Design

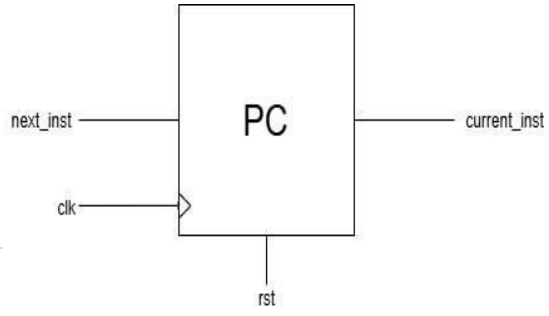
This section explains how the processor is designed and organized. The design focuses on how data moves inside the processor and how control signals are generated to execute instructions correctly. Than it will explain the addition of pipelining and methods to overcome the hazrds. The processor is designed to support the RV32I instruction set and handle different types of instructions efficiently.

2.1. Designing Datapath

The datapath shows how data flows through different hardware components inside the processor. It includes all the units that take part in instruction execution. Here is he description the block required for the datapath of RV32I processor:

2.1.1. Program Counter

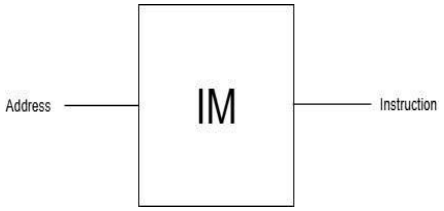
The program counter keeps the address of the instruction that the processor is currently running. After completing that instruction, the PC moves ahead by four bytes to reach the next one. If the instruction is a branch or a jump, the PC goes to a new address instead of just moving forward. In simple words,



the PC decides which instruction the processor should run next.

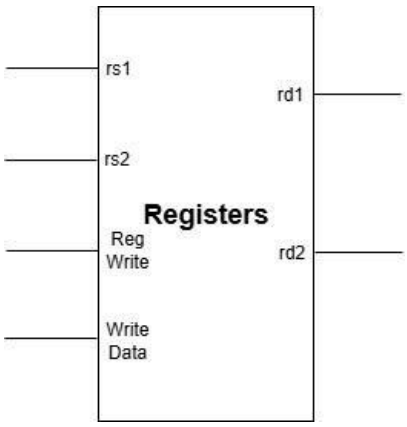
2.1.2. Instruction Memory

The Instruction Memory stores all the instructions of the program. It takes the address from the PC and outputs the corresponding instruction to be executed by the processor. The Instruction Memory (IM) has one input and one output. The input is the Address, which comes from the Program Counter (PC), and the output is the Instruction, which is sent to the processor for execution. When an address is given, the IM fetches the instruction stored at that location and outputs it.



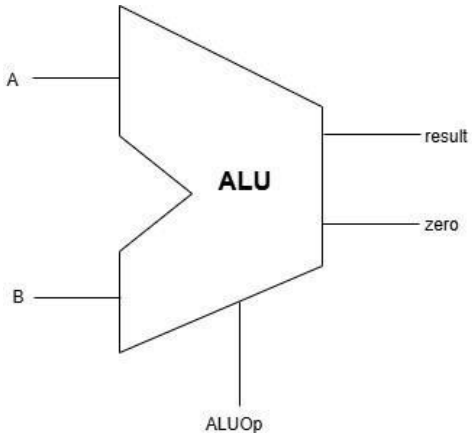
2.1.3. Registers

Registers are small, fast storage units inside the processor that hold data and instructions temporarily during execution. They allow quick access to values needed for arithmetic, logic, and control operations. The register file has 32 general-purpose registers (x0–x31), each 32 bits wide. It has two read ports and one write port, allowing the processor to read two operands and write one result in the same cycle. The input ports include rs1, rs2, and rd (5-bit register addresses), rd_data (32-bit write data), RegWrite (control signal), and clk (clock). The output ports are rs1_data and rs2_data, which provide the values stored in the selected registers. Internally, the register file is built from 32 arrays of D flip-flops for data storage, a 5-to-32 decoder to select the register to write into, and two 32-to-1 multiplexers to select which registers to read. Reads are asynchronous, while writes occur synchronously on the rising edge of the clock. Register x0 is permanently tied to zero and cannot be modified.



2.1.4. ALU

The Arithmetic Logic Unit (ALU) in a RISC-V single-cycle processor is responsible for performing all arithmetic and logical operations required by instructions. It takes two 32-bit input operands (A and B) and produces a 32-bit output (Result) along with status signals such as Zero, which indicates whether the output is zero. The operation performed by the ALU is controlled by the ALUControl signal, which selects among functions like addition, subtraction, AND, OR, XOR, shift, and set-less-than. Internally, the ALU is made up of combinational logic circuits including adders, logic gates, and shifters. For arithmetic operations, it uses a 32-bit adder/subtractor, while logical operations are handled by AND, OR, XOR gates, and shift operations are managed by barrel shifters. The ALU's design is purely combinational, meaning it produces the output instantly after the inputs or control signals change, without depending on the clock.



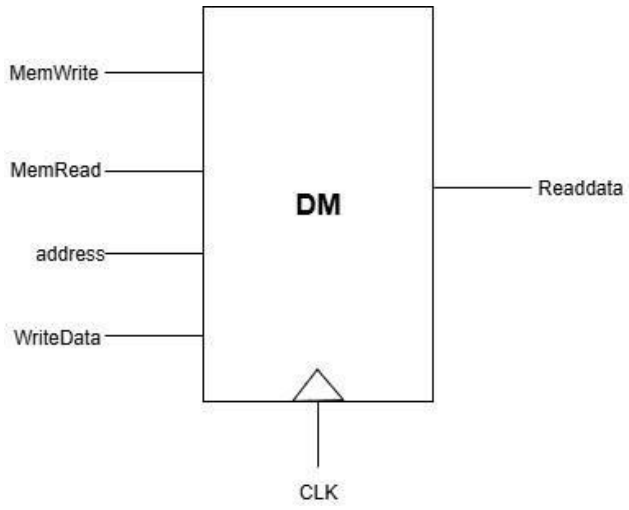
Except this we used two ALUs in the RV32I processor:

PC ALU: The PC ALU adds the current program counter (PC) value with 4 to calculate the address of the next instruction in memory.

Branch ALU: The Branch ALU is used to calculate the target address for branch instructions. It adds the immediate value from the instruction to the current PC to find where the program should jump if the branch condition is true. It also evaluates branch conditions, such as checking if two registers are equal or if one is greater than another, to decide whether the branch should be taken or not.

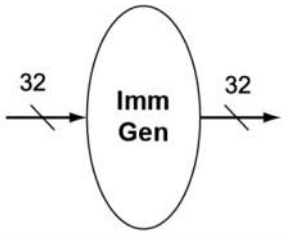
2.1.5. Data Memory

The Data Memory (DM) in a RISC-V single-cycle processor is used to store and access data during program execution, mainly for load and store instructions. It takes a 32-bit address input (usually from the ALU), a 32-bit write data input, and control signals MemRead and MemWrite to determine whether to read from or write to memory. The read data output provides the value fetched from the specified memory location. Internally, the data memory is built using RAM cells or register arrays, organized as a set of memory words, each 32 bits wide. When MemWrite is enabled, the data is written to the specified address on the rising edge of the clock (synchronous write). When MemRead is enabled, the data is read asynchronously from the given address and output immediately. In RISC-V, memory is byte-addressable, meaning each address refers to an 8-bit byte, and the processor can perform word (32-bit), half-word (16-bit), or byte (8-bit) accesses depending on the instruction type.



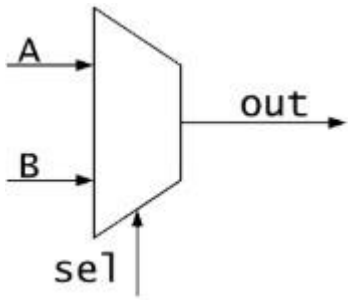
2.1.6. Immediate Generator

The immediate generator extracts and formats immediate values from the instruction. These values are used in operations such as arithmetic with constants, memory address calculation, and branch decisions.



2.1.7. Multiplexers

A multiplexer is a basic selection unit used in the datapath. We used 2x1 muxes. It has two input signals and one output signal. A control signal decides which input is passed to the output. If the control signal is low, the multiplexer selects the first input. If the control signal is high, it selects the second input. This allows the processor to choose between different data sources based on



the instruction being executed.
By combining these blocks we get the Datapath given below:

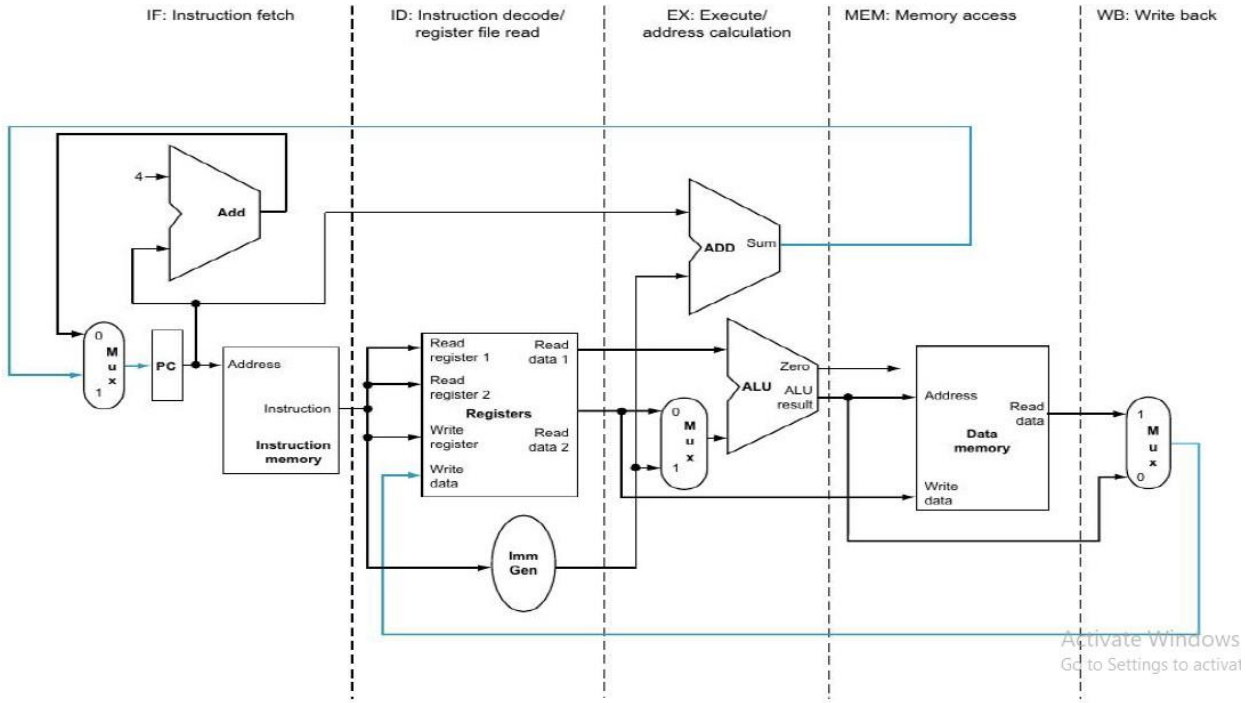
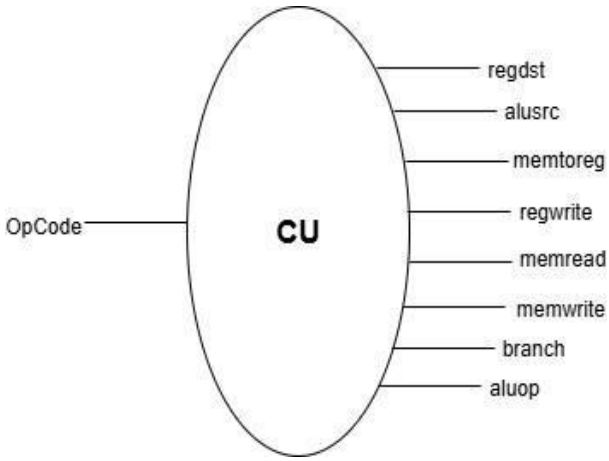


Figure 2: Datapath of RV32I

2.2. Control Path

The control path is responsible for generating signals that guide how the datapath operates for each instruction. It ensures that the right operations are performed at the right time. The main elements of the control path in our RV32I processor are:

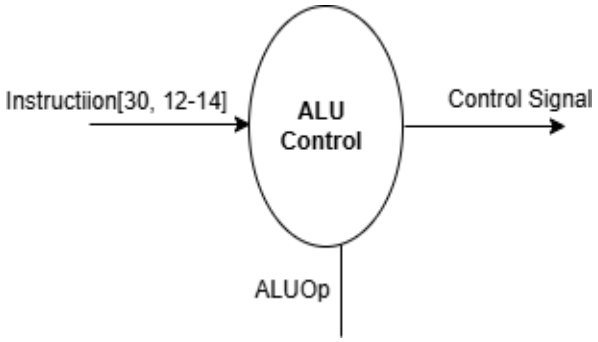
2.2.1. Main Control Unit



The Control Unit (CU) in a RISC-V single-cycle processor is responsible for decoding the instruction and generating the necessary control signals to direct the flow of data through the processor. It takes the opcode (and sometimes the `funct3` and `funct7` fields) from the instruction as input and produces signals such as RegWrite, MemRead, MemWrite, Branch, ALUSrc, MemToReg, and ALUOp. These signals determine how other components like the ALU, Register File, and Data Memory operate for a given instruction. Internally, the control unit is made up of combinational logic circuits (mainly decoders and logic gates) that map instruction fields to the correct control outputs. It ensures that each instruction type: R-type, I-type, S-type, B-type, U-type, and J-type, activates the appropriate datapath elements to perform its function. The CU operates without a clock, meaning its output changes immediately when the instruction input changes.

2.2.2. ALU Control Unit

The ALU control unit determines the specific operation the ALU should perform. It uses instruction fields and signals from the main control unit to select the correct ALU operation, like addition, subtraction, logical AND, OR, or shift operations.



Adding these blocks with in the datapath completes the RV32I single cycle processor.

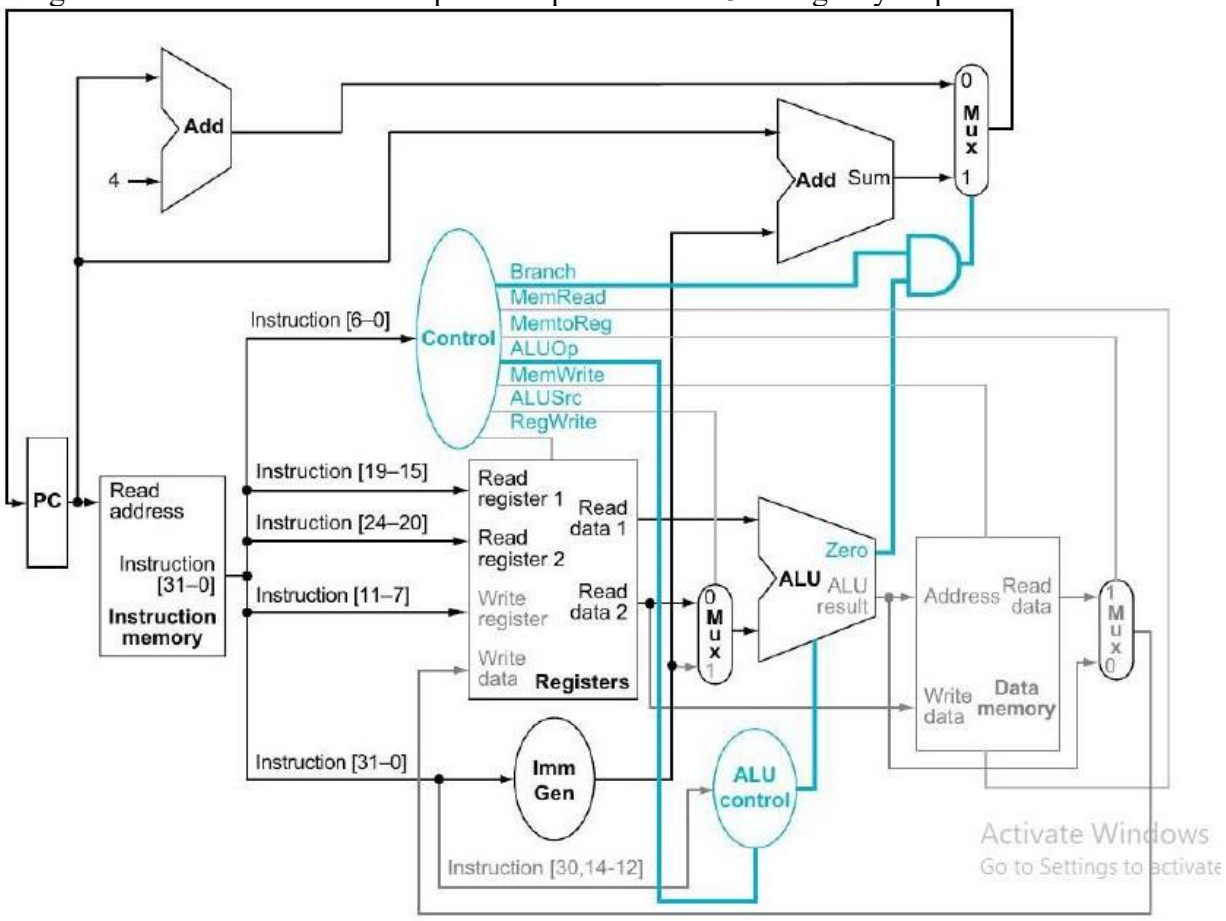


Figure 3: RV32I Processor with Control Path

2.3. Adding Pipeline Registers

Until now, our RV32I processor supports all 47 instructions and executes them correctly. To increase efficiency, we added pipeline registers between the stages. These registers store all the necessary information for an instruction as it moves from one stage to the next, keeping each stage independent and allowing multiple instructions to be in progress at the same time.

The main pipeline registers and their details are:

2.3.1. IF/ID Register

This register is placed between the Instruction Fetch (IF) and Instruction Decode (ID) stages. It holds the instruction fetched from memory and the program counter (PC) value. This allows the decode stage to work on the instruction while the fetch stage moves on to the next instruction.

2.3.2. ID/EX Register

The ID/EX register is placed between the Instruction Decode and Execute stages. It contains the decoded instruction fields, the values read from the source registers, immediate values extracted from the instruction, and all the control signals needed for execution, such as ALU operation selection, memory read or write, and branch control information. With this data stored, the execute stage has everything it needs to perform the required operations, independently of the decode stage.

2.3.3. EX/MEM Register

The EX/MEM register is located between the Execute and Memory Access stages. It stores the results calculated by the ALU, the target address for memory operations or branches, the data that needs to be written to memory for store instructions, and the relevant control signals for memory access and write-back. This ensures that the memory stage can carry out its operations accurately without waiting for other stages.

2.3.4. MEM/WB Register

The MEM/WB register sits between the Memory Access and Write Back stages. It contains the data read from memory for load instructions, the ALU results for arithmetic and logical operations, and control signals that indicate whether to write back to a register and which register should be updated. By keeping this information, the write-back stage can update the register file correctly, regardless of what the other stages are doing.

After adding these register now our 5-stage pipelined processor is complete as below:

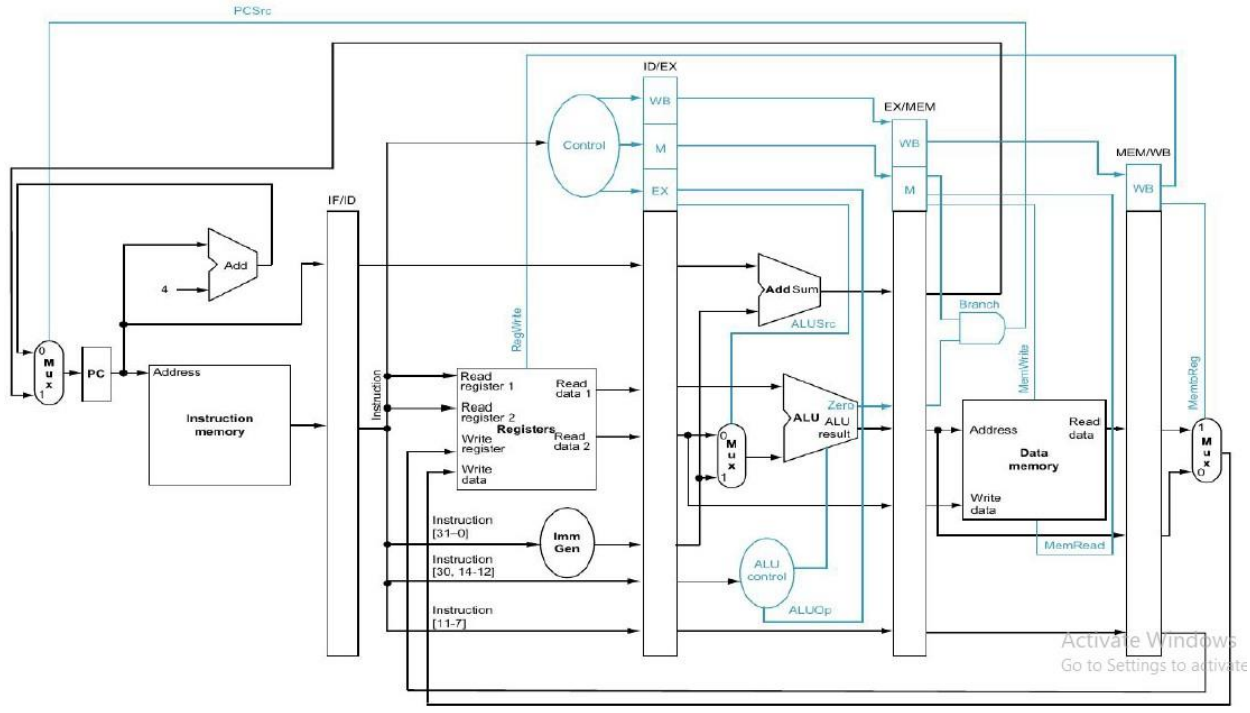


Figure 4: 5-stage RB32I Pipelined Processor

2.4. Working on Hazard Control

As mentioned earlier pipelining the processor causes data, control and structural hazards. Now let's talk about what we did to resolve these issues:

2.4.1. Data Hazards Solution

For resolving data hazards, a Forwarding Unit was introduced in the processor. This unit continuously monitors the destination registers of instructions in the EX/MEM and MEM/WB pipeline stages and compares them with the source registers of the instruction in the ID/EX stage. When a match is detected, the forwarding unit generates control signals that select the most recent operand values. To support this mechanism, two multiplexers were added before the ALU inputs. These multiplexers allow the ALU to receive data either directly from the register file or forwarded data from later pipeline stages. As a result, the execution stage can operate on the correct and most up-to-date values without waiting for the write-back stage, thereby reducing pipeline stalls and improving overall performance.

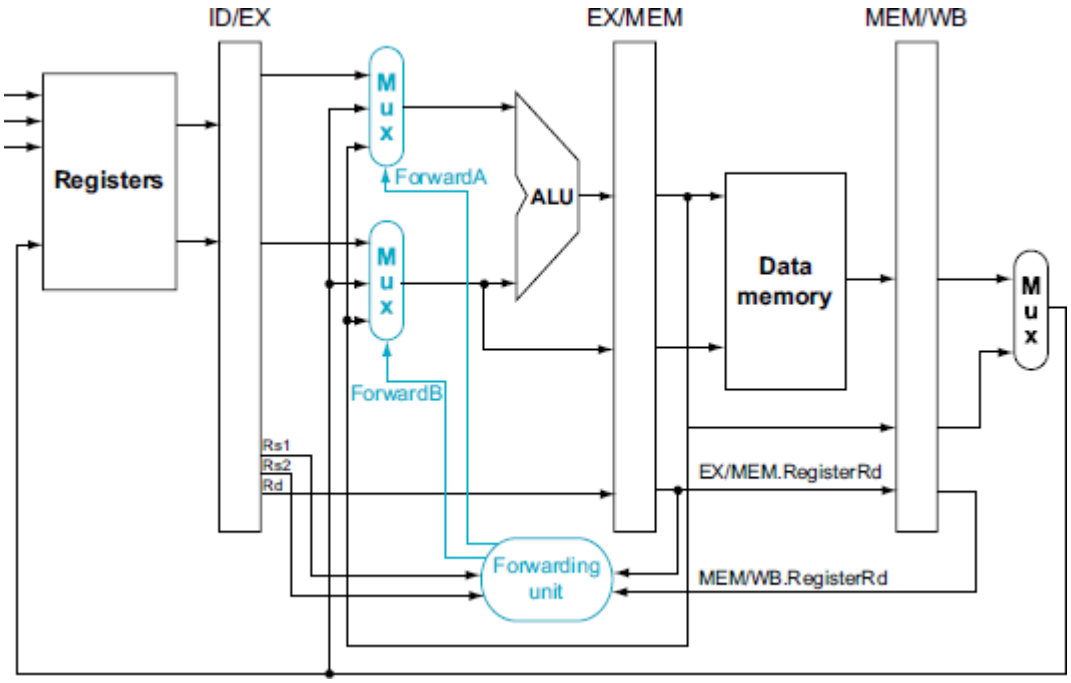


Figure 5: Addition of Forwarding Unit

Although forwarding resolves most read-after-write dependencies, it cannot handle load-use hazards where a load instruction is immediately followed by an instruction that uses the loaded data. For example:

```
lw   x5, 0(x1)
add  x6, x5, x2
```

In this example, the first instruction loads a value from memory into register x5. The second instruction immediately uses x5 as a source operand for an addition operation. Since the load instruction retrieves data from memory, the loaded value is not available until the end of the memory access stage. Forwarding alone cannot supply this data in time for the execution stage of the following instruction to address this issue, a Hazard Detection Unit was added. This unit detects when an instruction in the ID/EX stage is performing a memory read and its destination register matches one of the source registers of the instruction in the IF/ID stage. Upon detection, the unit temporarily disables program counter updates and IF/ID pipeline register writes, effectively stalling the pipeline for one clock cycle.

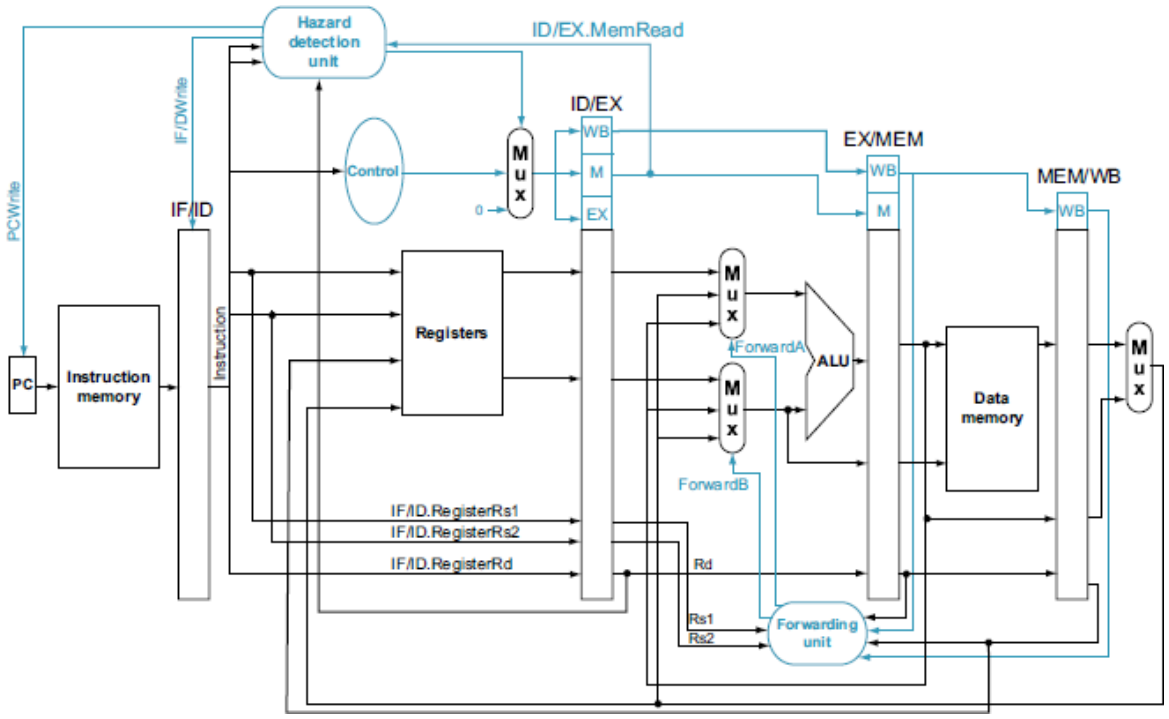


Figure 6: RV32I Pipelined Processor with Hazard Detection Unit

2.4.2. Control Hazards Solution

For control hazards introduced by branch and jump instructions, pipeline flushing logic was added. Once a branch or jump decision is resolved and the control transfer is taken, the incorrectly fetched instruction in the IF/ID pipeline register is flushed. The same Control Flush Unit is used to insert a NOP by clearing control signals, while the program counter is updated with the correct branch or jump target address. This design does not use branch prediction and instead relies on resolving control hazards using a stall-and-flush mechanism.

2.4.3. Structural Hazard Solution

Structural hazards occur when two or more pipeline stages need the same hardware resource at the same time. In your processor, this issue is avoided by design because instruction memory and data memory are implemented as separate and independent blocks. As a result, instruction fetch can happen in parallel with data read or write operations without any resource conflict.

Now, our 5-stage pipelined RV32I processor is complete with hazards control.

Code

ALU

```
module ALU(in1,in2,alu_result,alu_op,zero);

input [31:0] in1, in2;
input [3:0] alu_op;

output reg [31:0] alu_result;
output reg zero;

always @(*) begin

    case(alu_op)

        // ADD
        4'b0000 : begin
            alu_result = in1 + in2;
            zero = (alu_result == 32'd0);
        end

  // SUB

        4'b0001 : begin
            alu_result = in1 - in2;
            zero = (alu_result == 32'd0);
        end


        // XOR
        4'b0010 : begin
            alu_result = in1 ^ in2;
            zero = (alu_result == 32'd0);
        end


        // OR

        4'b0011 : begin
            alu_result = in1 | in2;
            zero = (alu_result == 32'd0);
        end


        // AND
```

```
        4'b0100 : begin
            alu_result = in1 & in2;
            zero = (alu_result == 32'd0);
        end


        // SLL

        4'b0101 : begin
            alu_result = in1 << in2[4:0];
            zero = (alu_result == 32'd0);
        end

        // SRL

        4'b0110 : begin
            alu_result = in1 >> in2[4:0];
            zero = (alu_result == 32'd0);
        end


        // SRA

        4'b0111 : begin
            alu_result = $signed(in1) >>> in2[4:0];
            zero = (alu_result == 32'd0);
        end

        // SLT

        4'b1000 : begin
            alu_result =
            ($signed(in1) < $signed(in2)) ? 32'd1 : 32'd0;

            zero = (alu_result == 32'd0);
        end


        // SLTU
            4'b1001 : begin
            alu_result =(in1 < in2) ? 32'd1 : 32'd0;

            zero = (alu_result == 32'd0);
        end
```

```
        // BEQ

        4'b1010 : begin
            alu_result = (in1==in2);

            if(in1 == in2)
                zero = 1'b1;
            else
                zero = 1'b0;
        end


        // BNE

        4'b1011 : begin
            alu_result = (in1 != in2);

            if(in1 != in2)
                zero = 1'b1;
            else
                zero = 1'b0;
        end

        // BLT

        4'b1100 : begin
            alu_result =
            ($signed(in1) < $signed(in2)) ? 32'd1 : 32'd0;

            zero = (alu_result == 32'd1);
        end

        // BGE
        4'b1101 : begin
            alu_result =
            ($signed(in1) >= $signed(in2)) ? 32'd1 : 32'd0;

            zero = (alu_result == 32'd1);
        end

        // BLTU
        4'b1110 : begin
            alu_result =
            (in1 < in2) ? 32'd1 : 32'd0;

            zero = (alu_result == 32'd1);
```

```
        end


        // BGEU

        4'b1111 : begin
            alu_result =
            (in1 >= in2) ? 32'd1 : 32'd0;

            zero = (alu_result == 32'd1);
        end


        // DEFAULT

        default : begin
            alu_result = 32'd0;
            zero = 1'b0;
        end

    endcase

end

endmodule
```

ALU_control

```
module ALU_Control(
    input [6:0] funct7,
    input [2:0] funct3,
    input [1:0] ALUOp,
    output reg [3:0] ALU_Ctrl
);

always @(*) begin

    casez ({ALUOp, funct7, funct3})

        // LOAD / STORE / ADDI
        12'b00_???????_???: ALU_Ctrl = 4'b0000;

        // R-TYPE

        12'b10_0000000_000: ALU_Ctrl = 4'b0000; // ADD
        12'b10_0100000_000: ALU_Ctrl = 4'b0001; // SUB
        12'b10_0000000_100: ALU_Ctrl = 4'b0010; // XOR
```

```
        12'b10_0000000_110: ALU_Ctrl = 4'b0011; // OR
        12'b10_0000000_111: ALU_Ctrl = 4'b0100; // AND
        12'b10_0000000_001: ALU_Ctrl = 4'b0101; // SLL
        12'b10_0000000_101: ALU_Ctrl = 4'b0110; // SRL
        12'b10_0100000_101: ALU_Ctrl = 4'b0111; // SRA
        12'b10_0000000_010: ALU_Ctrl = 4'b1000; // SLT
        12'b10_0000000_011: ALU_Ctrl = 4'b1001; // SLTU

        // BRANCH

        12'b01_???????_000: ALU_Ctrl = 4'b1010; // BEQ
        12'b01_???????_001: ALU_Ctrl = 4'b1011; // BNE
        12'b01_???????_100: ALU_Ctrl = 4'b1100; // BLT
        12'b01_???????_101: ALU_Ctrl = 4'b1101; // BGE
        12'b01_???????_110: ALU_Ctrl = 4'b1110; // BLTU
        12'b01_???????_111: ALU_Ctrl = 4'b1111; // BGEU
        default: ALU_Ctrl = 4'b0000;
    endcase
end
endmodule
```

AND Gate

```
module AND_gate(zero,branch,out);
  input zero,branch;
  output out;
  assign out = zero & branch;
endmodule
```

Control Unit

```
module
control_unit(instruction,Branch,MemRead,MemtoReg,AluOp,MemWrite,ALUSrc,RegWrite,Ju
mp);
        input [6:0] instruction;
        output reg Jump,Branch,MemRead,MemtoReg,MemWrite,ALUSrc,RegWrite;
        output reg [1:0] AluOp;

always@(*)
begin
        case(instruction)
                7'b0110011 :
{ALUSrc,MemtoReg,RegWrite,MemRead,MemWrite,Branch,Jump,AluOp} =
9'b0010000_10;    // R-type
                7'b0010011 :
{ALUSrc,MemtoReg,RegWrite,MemRead,MemWrite,Branch,Jump,AluOp} =
9'b1010000_00;     // I-ALU
                7'b0000011 :
```

```
{ALUSrc,MemtoReg,RegWrite,MemRead,MemWrite,Branch,Jump,AluOp} =
9'b1111000_00;     // I-Load
                7'b0100011 :
{ALUSrc,MemtoReg,RegWrite,MemRead,MemWrite,Branch,Jump,AluOp} =
9'b1000100_00;     // S-type
                7'b1100011 :
{ALUSrc,MemtoReg,RegWrite,MemRead,MemWrite,Branch,Jump,AluOp} =
9'b0000010_01;     // B-type
                7'b1101111 :
{ALUSrc,MemtoReg,RegWrite,MemRead,MemWrite,Branch,Jump,AluOp} =
9'b0010001_00;     // JAL
                7'b1100111 :
{ALUSrc,MemtoReg,RegWrite,MemRead,MemWrite,Branch,Jump,AluOp} =
9'b1010001_00;     // JALR
                7'b0110111 :
{ALUSrc,MemtoReg,RegWrite,MemRead,MemWrite,Branch,Jump,AluOp} =
9'b1010000_00;     // LUI
                7'b0010111 :
{ALUSrc,MemtoReg,RegWrite,MemRead,MemWrite,Branch,Jump,AluOp} =
9'b1010000_00;     // AUIPC
    default :    {ALUSrc,MemtoReg,RegWrite,MemRead,MemWrite,Branch,Jump,AluOp} =
9'b000000000;
        endcase
end
endmodule
```

Data Memory

```
module data_memory(
    input clk,
    input reset,
    input mem_write,
    input mem_read,
    input [2:0] funct3,       // operation selector
    input [31:0] Address,
    input [31:0] write_data,
    output reg [31:0] read_data
);

    // 1 KB memory = 256 words
    reg [31:0] data_mem [0:255];

    integer i;

    wire [7:0] byte_offset;
    assign byte_offset = Address[1:0];
```

```
// WRITE OPERATIONS
always @(posedge clk or posedge reset) begin
    if (reset) begin
        for (i = 0; i < 256; i = i + 1)
            data_mem[i] <= 32'b0;
    end

    else if (mem_write) begin

        case (funct3)

            // SW : Store Word
            // funct3 = 010

            3'b010: begin
                data_mem[Address[9:2]] <= write_data;
            end

            // SB : Store Byte
            // funct3 = 000

            3'b000: begin
                case (byte_offset)

                    2'b00:
                        data_mem[Address[9:2]][7:0]
                            <= write_data[7:0];

                    2'b01:
                        data_mem[Address[9:2]][15:8]
                            <= write_data[7:0];

                    2'b10:
                        data_mem[Address[9:2]][23:16]
                            <= write_data[7:0];

                    2'b11:
                        data_mem[Address[9:2]][31:24]
                            <= write_data[7:0];

                endcase
            end


            // SH : Store Half Word
```

```
            // funct3 = 001

            3'b001: begin

                case (Address[1])

                    1'b0:
                        data_mem[Address[9:2]][15:0]
                            <= write_data[15:0];

                    1'b1:
                        data_mem[Address[9:2]][31:16]
                            <= write_data[15:0];

                endcase
            end

        endcase
    end
end

// READ OPERATIONS

always @(*) begin

    if (mem_read) begin

        case (funct3)

              // LW : Load Word
            // funct3 = 010

            3'b010: begin
                read_data = data_mem[Address[9:2]];
            end


            // LB : Load Byte
            // funct3 = 000
             3'b000: begin

                case (byte_offset)

                    2'b00:
                        read_data =
                        {{24{data_mem[Address[9:2]][7]}},
```

```
                        data_mem[Address[9:2]][7:0]};

                    2'b01:
                        read_data =
                        {{24{data_mem[Address[9:2]][15]}},
                          data_mem[Address[9:2]][15:8]};

                    2'b10:
                        read_data =
                        {{24{data_mem[Address[9:2]][23]}},
                          data_mem[Address[9:2]][23:16]};

                    2'b11:
                        read_data =
                        {{24{data_mem[Address[9:2]][31]}},
                          data_mem[Address[9:2]][31:24]};

                endcase
            end

            // LH : Load Half Word
            // funct3 = 00

            3'b001: begin

                case (Address[1])

                    1'b0:
                        read_data =
                        {{16{data_mem[Address[9:2]][15]}},
                          data_mem[Address[9:2]][15:0]};

                    1'b1:
                        read_data =
                        {{16{data_mem[Address[9:2]][31]}},
                          data_mem[Address[9:2]][31:16]};

                endcase
            end

            default:
                read_data = 32'b0;

        endcase

    end
```

```
        else
            read_data = 32'b0;

    end

endmodule
```

Register File

```
module Reg_File(clk, reset, Regwrite, Rs1, Rs2, Rd, Write_data, read_data1,
read_data2);
    input         clk, reset, Regwrite;
    input  [4:0] Rs1, Rs2, Rd;
    input  [31:0] Write_data;
    output [31:0] read_data1, read_data2;

    integer k;
    reg [31:0] Registers[31:0];

    // Synchronous write
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            for (k = 0; k < 32; k = k + 1)
                Registers[k] <= 32'b0;
        end else if (Regwrite && (Rd != 5'b0)) begin
            Registers[Rd] <= Write_data;
        end
    end
    assign read_data1 = (Regwrite && (Rd != 5'b0) && (Rd == Rs1)) ? Write_data :
Registers[Rs1];
    assign read_data2 = (Regwrite && (Rd != 5'b0) && (Rd == Rs2)) ? Write_data :
Registers[Rs2];

endmodule
```

Program Counter

```
module program_counter(clk,reset,pc_write,pc_in,pc_out);
        input clk, reset, pc_write;
        input [31:0] pc_in;
        output reg [31:0] pc_out;

        always@(posedge clk or posedge reset)
        begin
                if (reset)
                        pc_out <= 32'b00;
                else if (pc_write)
```

```
                        pc_out <= pc_in;
                else
                        pc_out <= pc_out;   // stall: hold PC
        end
endmodule
```

PC + offset Adder

```
module branch_adder(
    input  [31:0] pc_out,
    input  [31:0] immediate_for_branch,
    output [31:0] adder_output
);
assign adder_output = pc_out + immediate_for_branch;
endmodule
```

PC + 4 Adder

```
module pc_adder(adder_in, adder_out);
        input [31:0] adder_in;
        output [31:0] adder_out;
        assign adder_out = adder_in + 4 ;
endmodule
```

OR Gate

```
module OR_gate(jump,AND_gate_output,OR_gate_result);
  input jump, AND_gate_output;
  output OR_gate_result;
  assign OR_gate_result = jump | AND_gate_output;
endmodule
```

3 Input Mux

```
module mux3 (
    input  [31:0] in0,   // 00: from register file (no forwarding)
    input  [31:0] in1,   // 10: forward from EX/MEM ALU result
    input  [31:0] in2,   // 01: forward from MEM/WB write-back
    input  [1:0]  sel,
    output [31:0] out
);
    assign out = (sel == 2'b10) ? in1 :
                 (sel == 2'b01) ? in2 :
                                  in0;
endmodule
```

2 Input Mux

```
module mux(
    input  [31:0] pc_out,
    input  [31:0] adder_output,
    input         sel,
    output [31:0] mux_output
```

```
);

assign mux_output = sel ? adder_output : pc_out;

endmodule
```

Instruction Memory

```
module instruction_memory(
    input  [31:0] address,
    output [31:0] instruction
);

reg [31:0] instruction_memory [0:31];

assign instruction = instruction_memory[address[31:2]];

integer i;
initial begin
    for (i = 0; i < 32; i = i + 1)
        instruction_memory[i] = 32'h00000013; // NOP

    // Lab-13 hazard checking program
    instruction_memory[0]  = 32'h00A00093; // addi x1, x0, 10
    instruction_memory[1]  = 32'h01400113; // addi x2, x0, 20
    instruction_memory[2]  = 32'h002081B3; // add  x3, x1, x2

    // RAW data hazard: forwarding required
    instruction_memory[3]  = 32'h40118233; // sub  x4, x3, x1
    instruction_memory[4]  = 32'h002272B3; // and  x5, x4, x2

    // Load-use hazard: one stall required
    instruction_memory[5]  = 32'h00002303; // lw   x6, 0(x0)
    instruction_memory[6]  = 32'h001303B3; // add  x7, x6, x1

    // Control hazard: taken branch, following two instructions flushed
    instruction_memory[7]  = 32'h00738663; // beq  x7, x7, TARGET (+12)
    instruction_memory[8]  = 32'h06400413; // addi x8, x0, 100  -> flushed
    instruction_memory[9]  = 32'h0C800493; // addi x9, x0, 200  -> flushed

    // TARGET:
    instruction_memory[10] = 32'h00208533; // add  x10, x1, x2
end

endmodule
```

Immediate Generator

```
module ImmGen(
    input [6:0] Opcode,
    input [31:0] instruction,
    output reg [31:0] ImmExt
);

always @(*) begin

    case(Opcode)

        // I-type ALU + LOAD
        7'b0010011,
        7'b0000011:
            ImmExt = {{20{instruction[31]}}, instruction[31:20]};

        // S-type
        7'b0100011:
            ImmExt = {{20{instruction[31]}}, instruction[31:25],
instruction[11:7]};

        // B-type
        7'b1100011:
            ImmExt = {{19{instruction[31]}}, instruction[31], instruction[7],
instruction[30:25], instruction[11:8], 1'b0};

        // U-type
        7'b0110111,
        7'b0010111:
            ImmExt = {instruction[31:12], 12'b0};

        // J-type
        7'b1101111:
            ImmExt = {{11{instruction[31]}}, instruction[31], instruction[19:12],
instruction[20], instruction[30:21], 1'b0};

        default:
            ImmExt = 32'b0;

    endcase

end

endmodule
```

IF/ID Register

```
module IF_ID_reg(
    input clk,
    input reset,
    input write_enable,
    input flush,
    input [31:0] IF_PC_in,
    input [31:0] IF_Program_mem,
    output reg [31:0] ID_PC_out,
    output reg [31:0] ID_Program_mem_out
);

always@(posedge clk or posedge reset)
begin
    if(reset || flush) begin
        ID_PC_out <= 32'b0;
        ID_Program_mem_out <= 32'h00000013; // NOP: addi x0,x0,0
    end
    else if(write_enable) begin
        ID_PC_out <= IF_PC_in;
        ID_Program_mem_out <= IF_Program_mem;
    end
    else begin
        ID_PC_out <= ID_PC_out;                  // stall: hold instruction
        ID_Program_mem_out <= ID_Program_mem_out;
    end
end
endmodule
```

ID/EXE Register

```
module ID_EXE_reg(
  input clk, reset, flush,
  input ID_Branch_in, ID_MemRead_in, ID_MemtoReg_in, ID_MemWrite_in, ID_ALUSrc_in,
ID_RegWrite_in, ID_Jump_in,
  input [1:0]  ID_AluOp_in,
  input [31:0] ID_PC_in,
  input [31:0] ID_Rd1_in, ID_Rd2_in,
  input [31:0] ID_ImmGen_in,
  input [31:0] ID_Instr_in,
  input [4:0]  ID_Rd_in,

  input [4:0]  ID_Rs1_in,
  input [4:0]  ID_Rs2_in,

  output reg EXE_Branch_out, EXE_MemRead_out, EXE_MemtoReg_out, EXE_MemWrite_out,
```

```
EXE_ALUSrc_out, EXE_RegWrite_out, EXE_Jump_out,
  output reg [1:0]  EXE_AluOp_out,
  output reg [31:0] EXE_PC_out,
  output reg [31:0] EXE_Rd1_out, EXE_Rd2_out,
  output reg [31:0] EXE_ImmGen_out,
  output reg [31:0] EXE_Instr_out,
  output reg [4:0]  EXE_Rd_out,

  output reg [4:0]  EXE_Rs1_out,
  output reg [4:0]  EXE_Rs2_out
);
  always@(posedge clk or posedge reset)
  begin
    if(reset || flush)
    begin
      EXE_Branch_out   <= 1'b0;
      EXE_MemRead_out  <= 1'b0;
      EXE_MemtoReg_out <= 1'b0;
      EXE_MemWrite_out <= 1'b0;
      EXE_ALUSrc_out   <= 1'b0;
      EXE_RegWrite_out <= 1'b0;
      EXE_Jump_out     <= 1'b0;
      EXE_AluOp_out    <= 2'b00;
      EXE_PC_out       <= 32'b0;
      EXE_Rd1_out      <= 32'b0;
      EXE_Rd2_out      <= 32'b0;
      EXE_ImmGen_out   <= 32'b0;
      EXE_Instr_out    <= 32'b0;
      EXE_Rd_out       <= 5'b0;
      EXE_Rs1_out      <= 5'b0;
      EXE_Rs2_out      <= 5'b0;
    end
    else
    begin
      EXE_Branch_out   <= ID_Branch_in;
      EXE_MemRead_out  <= ID_MemRead_in;
      EXE_MemtoReg_out <= ID_MemtoReg_in;
      EXE_MemWrite_out <= ID_MemWrite_in;
      EXE_ALUSrc_out   <= ID_ALUSrc_in;
      EXE_RegWrite_out <= ID_RegWrite_in;
      EXE_Jump_out     <= ID_Jump_in;
      EXE_AluOp_out    <= ID_AluOp_in;
      EXE_PC_out       <= ID_PC_in;
      EXE_Rd1_out      <= ID_Rd1_in;
      EXE_Rd2_out      <= ID_Rd2_in;
      EXE_ImmGen_out   <= ID_ImmGen_in;
```

```
      EXE_Instr_out    <= ID_Instr_in;
      EXE_Rd_out       <= ID_Rd_in;
      EXE_Rs1_out      <= ID_Rs1_in;
      EXE_Rs2_out      <= ID_Rs2_in;
    end
  end
endmodule
```

EXE/MEM

```
module EXE_MEM_reg(
  input clk,reset,flush,
        input
EXE_Branch_in,EXE_MemRead_in,EXE_MemtoReg_in,EXE_MemWrite_in,EXE_RegWrite_in,EXE_J
ump_in,
  input [31:0] EXE_adder_result_in,
  input [31:0] EXE_ALU_result_in,
  input [4:0] EXE_read_data_in,
    input [2:0] EXE_funct3_in,
  input [31:0] EXE_write_data_in,
  input EXE_zero_in,

  output reg
MEM_Branch_out,MEM_MemRead_out,MEM_MemtoReg_out,MEM_MemWrite_out,MEM_RegWrite_out,
MEM_Jump_out,
        output reg [31:0] MEM_adder_result_out,
        output reg [31:0] MEM_ALU_result_out,
        output reg [4:0] MEM_read_data_out,
        output reg [2:0] MEM_funct3_out,
        output reg [31:0] MEM_write_data_out,
  output reg MEM_zero_out
  );
always@(posedge clk or posedge reset)
  begin
  if (reset || flush)
  begin

    MEM_Branch_out <= 1'b0;
    MEM_MemRead_out <= 1'b0;
    MEM_MemtoReg_out <= 1'b0;
    MEM_MemWrite_out <= 1'b0;
    MEM_RegWrite_out <= 1'b0;
    MEM_Jump_out <= 1'b0;
    MEM_adder_result_out <= 32'b00;
    MEM_ALU_result_out <= 32'b00;
    MEM_read_data_out <= 5'b00;
        MEM_funct3_out <= 3'b010;
```

```
    MEM_write_data_out <= 32'b00;
    MEM_zero_out <= 1'b0;
  end
  else
  begin
    MEM_Branch_out <= EXE_Branch_in;
                MEM_MemRead_out <= EXE_MemRead_in;
                MEM_MemtoReg_out <= EXE_MemtoReg_in;
                MEM_MemWrite_out <= EXE_MemWrite_in;
                MEM_RegWrite_out <= EXE_RegWrite_in;
                MEM_Jump_out <= EXE_Jump_in;
                MEM_adder_result_out <= EXE_adder_result_in;
                MEM_ALU_result_out <= EXE_ALU_result_in;
                MEM_read_data_out <= EXE_read_data_in;
                MEM_funct3_out <= EXE_funct3_in;
                MEM_write_data_out <= EXE_write_data_in;
    MEM_zero_out <= EXE_zero_in;
  end
  end
endmodule
```

MEM/WB Reg

```
module MEM_WB_reg (
    input         clk, reset,
    // WB control group only
    input         MEM_MemtoReg_in,
    input         MEM_RegWrite_in,
    // Data
    input  [31:0] MEM_ALU_result_in,
    input  [4:0]  MEM_Rd_in,
    input  [31:0] MEM_read_data_in,
    // Outputs
    output reg        WB_MemtoReg_out,
    output reg        WB_RegWrite_out,
    output reg [31:0] WB_ALU_result_out,
    output reg [4:0]  WB_Rd_out,
    output reg [31:0] WB_read_data_out
);
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            WB_MemtoReg_out   <= 1'b0;
            WB_RegWrite_out   <= 1'b0;
            WB_ALU_result_out <= 32'b0;
            WB_Rd_out         <= 5'b0;
            WB_read_data_out  <= 32'b0;
        end else begin
```

```
            WB_MemtoReg_out   <= MEM_MemtoReg_in;
            WB_RegWrite_out   <= MEM_RegWrite_in;
            WB_ALU_result_out <= MEM_ALU_result_in;
            WB_Rd_out         <= MEM_Rd_in;
            WB_read_data_out  <= MEM_read_data_in;
        end
    end
endmodule
```

Hazard Detection Unit

```
module hazard_detection_unit(
    input        ID_EX_MemRead,
    input  [4:0] ID_EX_Rd,
    input  [4:0] IF_ID_Rs1,
    input  [4:0] IF_ID_Rs2,
    input        PCSrc,          // branch/jump taken signal

    output reg   PCWrite,
    output reg   IF_ID_Write,
    output reg   IF_ID_Flush,
    output reg   ID_EX_Flush,
    output reg   EX_MEM_Flush,
    output reg   Stall
);

always @(*) begin
    // Normal case: pipeline runs freely
    PCWrite     = 1'b1;
    IF_ID_Write = 1'b1;
    IF_ID_Flush = 1'b0;
    ID_EX_Flush = 1'b0;
    EX_MEM_Flush= 1'b0;
    Stall       = 1'b0;

    // Control hazard has highest priority
    if (PCSrc) begin
        PCWrite      = 1'b1;
        IF_ID_Write  = 1'b1;
        IF_ID_Flush  = 1'b1;
        ID_EX_Flush  = 1'b1;
        EX_MEM_Flush = 1'b1;
        Stall        = 1'b0;
    end

    // Load-use hazard:
    // If instruction in EX is load and next instruction in ID uses same rd,
```

```
    // hold PC and IF/ID, and convert ID/EX control to NOP for one cycle.
    else if (ID_EX_MemRead && (ID_EX_Rd != 5'b00000) &&
             ((ID_EX_Rd == IF_ID_Rs1) || (ID_EX_Rd == IF_ID_Rs2))) begin
        PCWrite     = 1'b0;
        IF_ID_Write = 1'b0;
        IF_ID_Flush = 1'b0;
        ID_EX_Flush = 1'b1;
        EX_MEM_Flush= 1'b0;
        Stall       = 1'b1;
    end
end
endmodule
```

Forwarding Unit

```
module forwarding_unit (
    input [4:0] ID_EX_Rs1,       // rs1 of instruction in EX
    input [4:0] ID_EX_Rs2,       // rs2 of instruction in EX

    input [4:0] EX_MEM_Rd,
    input       EX_MEM_RegWrite,

    input [4:0] MEM_WB_Rd,
    input       MEM_WB_RegWrite,

    output reg [1:0] ForwardA,   // controls ALU input 1
    output reg [1:0] ForwardB    // controls ALU input 2
);
    always @(*) begin
        if (EX_MEM_RegWrite && (EX_MEM_Rd != 5'b0) && (EX_MEM_Rd == ID_EX_Rs1))
            ForwardA = 2'b10;
        else if (MEM_WB_RegWrite && (MEM_WB_Rd != 5'b0) && (MEM_WB_Rd ==
ID_EX_Rs1))
            ForwardA = 2'b01;
        else
            ForwardA = 2'b00;

        if (EX_MEM_RegWrite && (EX_MEM_Rd != 5'b0) && (EX_MEM_Rd == ID_EX_Rs2))
            ForwardB = 2'b10;
        else if (MEM_WB_RegWrite && (MEM_WB_Rd != 5'b0) && (MEM_WB_Rd ==
ID_EX_Rs2))
            ForwardB = 2'b01;
        else
            ForwardB = 2'b00;
    end

endmodule
```

Top Module

```
module tope (
    input clk,
    input reset
);

// ============================================================
//  IF Stage wires
// ============================================================
wire [31:0] pc_out;
wire [31:0] pc_in;
wire [31:0] pc_plus4;
wire [31:0] instruction;

// ============================================================
//  IF/ID pipeline register outputs  (ID stage inputs)
// ============================================================
wire [31:0] ID_PC_out;
wire [31:0] ID_Prog_mem_out;

wire [6:0] ID_opcode = ID_Prog_mem_out[6:0];
wire [4:0] ID_rd     = ID_Prog_mem_out[11:7];
wire [4:0] ID_rs1    = ID_Prog_mem_out[19:15];
wire [4:0] ID_rs2    = ID_Prog_mem_out[24:20];

// ============================================================
//  ID Stage wires
// ============================================================
wire        branch, mem_read, mem_to_reg;
wire [1:0]  alu_op;
wire        mem_write, alu_src, reg_write, jump;

wire [31:0] read_data1, read_data2;
wire [31:0] imm_ext;
wire [31:0] write_data_reg;

// ============================================================
//  ID/EXE pipeline register outputs  (EXE stage inputs)
// ============================================================
wire        EXE_Branch,  EXE_MemRead,  EXE_MemtoReg;
wire        EXE_MemWrite, EXE_ALUSrc,  EXE_RegWrite, EXE_Jump;
wire [1:0]  EXE_AluOp;
wire [31:0] EXE_PC, EXE_Rd1, EXE_Rd2, EXE_ImmGen, EXE_Instr;
wire [4:0]  EXE_Rd;
wire [4:0]  EXE_Rs1, EXE_Rs2;   // for forwarding unit
```

```
wire [6:0]  EXE_funct7 = EXE_Instr[31:25];
wire [2:0]  EXE_funct3 = EXE_Instr[14:12];

// ============================================================
//  EXE Stage wires
// ============================================================
wire [3:0]  alu_ctrl;
wire [31:0] alu_in1;        // ALU input 1 (after ForwardA mux)
wire [31:0] alu_in2_fwd;    // ALU input 2 after ForwardB mux (before ALUSrc mux)
wire [31:0] alu_in2;        // ALU input 2 final (after ALUSrc mux)
wire [31:0] alu_result;
wire        zero;
wire [31:0] branch_target;

// Forwarding control
wire [1:0]  ForwardA, ForwardB;

// Hazard detection control
wire PCWrite;
wire IF_ID_Write;
wire IF_ID_Flush;
wire ID_EX_Flush;
wire EX_MEM_Flush;
wire Stall;
wire pc_src;

// ============================================================
//  EXE/MEM pipeline register outputs  (MEM stage inputs)
// ============================================================
wire        MEM_Branch,  MEM_MemRead,  MEM_MemtoReg;
wire        MEM_MemWrite, MEM_RegWrite, MEM_Jump;
wire [31:0] MEM_adder_result;
wire [31:0] MEM_ALU_result;
wire [4:0]  MEM_Rd;
wire [2:0]  MEM_funct3;
wire [31:0] MEM_write_data;
wire        MEM_zero;

wire [31:0] read_data_mem;
wire        and_out;
wire        or_out;

// ============================================================
//  MEM/WB pipeline register outputs  (WB stage inputs)
// ============================================================
```

```
wire        WB_MemtoReg, WB_RegWrite;
wire [31:0] WB_ALU_result;
wire [4:0]  WB_Rd;
wire [31:0] WB_read_data;

// ============================================================
//  WB Stage — MemtoReg MUX (2-input)
// ============================================================
assign write_data_reg = WB_MemtoReg ? WB_read_data : WB_ALU_result;

// ============================================================
//  Module Instantiations
// ============================================================

// ── IF Stage ──────────────────────────────────────────────
program_counter PC (
    .clk    (clk),
    .reset  (reset),
    .pc_write (PCWrite),
    .pc_in  (pc_in),
    .pc_out (pc_out)
);

pc_adder PC_PLUS4 (
    .adder_in  (pc_out),
    .adder_out (pc_plus4)
);

instruction_memory IMEM (
    .address     (pc_out),
    .instruction (instruction)
);

// ── IF/ID Register ────────────────────────────────────────
IF_ID_reg IF_ID (
    .clk                (clk),
    .reset              (reset),
    .write_enable       (IF_ID_Write),
    .flush              (IF_ID_Flush),
    .IF_PC_in           (pc_out),
    .IF_Program_mem     (instruction),
    .ID_PC_out          (ID_PC_out),
    .ID_Program_mem_out (ID_Prog_mem_out)
);

// ── ID Stage ──────────────────────────────────────────────
```

```
control_unit CU (
    .instruction (ID_opcode),
    .Branch      (branch),
    .MemRead     (mem_read),
    .MemtoReg    (mem_to_reg),
    .AluOp       (alu_op),
    .MemWrite    (mem_write),
    .ALUSrc      (alu_src),
    .RegWrite    (reg_write),
    .Jump        (jump)
);

Reg_File RF (
    .clk        (clk),
    .reset      (reset),
    .Regwrite   (WB_RegWrite),
    .Rs1        (ID_rs1),
    .Rs2        (ID_rs2),
    .Rd         (WB_Rd),
    .Write_data (write_data_reg),
    .read_data1 (read_data1),
    .read_data2 (read_data2)
);

ImmGen IMM (
    .Opcode      (ID_opcode),
    .instruction (ID_Prog_mem_out),
    .ImmExt      (imm_ext)
);

// Hazard Detection Unit
hazard_detection_unit HDU (
    .ID_EX_MemRead (EXE_MemRead),
    .ID_EX_Rd      (EXE_Rd),
    .IF_ID_Rs1     (ID_rs1),
    .IF_ID_Rs2     (ID_rs2),
    .PCSrc         (pc_src),
    .PCWrite       (PCWrite),
    .IF_ID_Write   (IF_ID_Write),
    .IF_ID_Flush   (IF_ID_Flush),
    .ID_EX_Flush   (ID_EX_Flush),
    .EX_MEM_Flush  (EX_MEM_Flush),
    .Stall         (Stall)
);

// ── ID/EXE Register ──────────────────────────────────────
```

```
ID_EXE_reg ID_EXE (
    .clk               (clk),
    .reset             (reset),
    .flush             (ID_EX_Flush),
    .ID_RegWrite_in    (reg_write),
    .ID_MemtoReg_in    (mem_to_reg),
    .ID_Jump_in        (jump),
    .ID_Branch_in      (branch),
    .ID_MemRead_in     (mem_read),
    .ID_MemWrite_in    (mem_write),
    .ID_ALUSrc_in      (alu_src),
    .ID_AluOp_in       (alu_op),
    .ID_PC_in          (ID_PC_out),
    .ID_Rd1_in         (read_data1),
    .ID_Rd2_in         (read_data2),
    .ID_ImmGen_in      (imm_ext),
    .ID_Instr_in       (ID_Prog_mem_out),
    .ID_Rd_in          (ID_rd),
    .ID_Rs1_in         (ID_rs1),
    .ID_Rs2_in         (ID_rs2),
    .EXE_RegWrite_out (EXE_RegWrite),
    .EXE_MemtoReg_out (EXE_MemtoReg),
    .EXE_Jump_out      (EXE_Jump),
    .EXE_Branch_out    (EXE_Branch),
    .EXE_MemRead_out   (EXE_MemRead),
    .EXE_MemWrite_out (EXE_MemWrite),
    .EXE_ALUSrc_out    (EXE_ALUSrc),
    .EXE_AluOp_out     (EXE_AluOp),
    .EXE_PC_out        (EXE_PC),
    .EXE_Rd1_out       (EXE_Rd1),
    .EXE_Rd2_out       (EXE_Rd2),
    .EXE_ImmGen_out    (EXE_ImmGen),
    .EXE_Instr_out     (EXE_Instr),
    .EXE_Rd_out        (EXE_Rd),
    .EXE_Rs1_out       (EXE_Rs1),
    .EXE_Rs2_out       (EXE_Rs2)
);

// ── EXE Stage ──────────────────────────────────────────────

// Forwarding Unit
forwarding_unit FWD (
    .ID_EX_Rs1        (EXE_Rs1),
    .ID_EX_Rs2        (EXE_Rs2),
    .EX_MEM_Rd        (MEM_Rd),
    .EX_MEM_RegWrite (MEM_RegWrite),
```

```
    .MEM_WB_Rd       (WB_Rd),
    .MEM_WB_RegWrite (WB_RegWrite),
    .ForwardA        (ForwardA),
    .ForwardB        (ForwardB)
);

ALU_Control ALU_CTRL (
    .funct7   (EXE_funct7),
    .funct3   (EXE_funct3),
    .ALUOp    (EXE_AluOp),
    .ALU_Ctrl (alu_ctrl)
);

// ForwardA MUX (3-input) — selects ALU operand 1
// 00 = register file, 10 = EX/MEM ALU result, 01 = MEM/WB write-back
mux3 MUX_FORWARD_A (
    .in0 (EXE_Rd1),         // no forwarding: value from register file
    .in1 (MEM_ALU_result),  // EX hazard:  forward from EX/MEM ALU result
    .in2 (write_data_reg),  // MEM hazard: forward from MEM/WB write-back
    .sel (ForwardA),
    .out (alu_in1)
);

// ForwardB MUX (3-input) — selects ALU operand 2 (before ALUSrc mux)
mux3 MUX_FORWARD_B (
    .in0 (EXE_Rd2),         // no forwarding: value from register file
    .in1 (MEM_ALU_result),  // EX hazard:  forward from EX/MEM ALU result
    .in2 (write_data_reg),  // MEM hazard: forward from MEM/WB write-back
    .sel (ForwardB),
    .out (alu_in2_fwd)
);

// ALUSrc MUX (2-input) — selects between forwarded Rd2 and immediate
assign alu_in2 = EXE_ALUSrc ? EXE_ImmGen : alu_in2_fwd;

ALU ALU_INST (
    .in1        (alu_in1),
    .in2        (alu_in2),
    .alu_op     (alu_ctrl),
    .alu_result (alu_result),
    .zero       (zero)
);

// Branch Target Adder
branch_adder BRANCH_ADDER (
    .pc_out               (EXE_PC),
```

```
    .immediate_for_branch (EXE_ImmGen),
    .adder_output         (branch_target)
);

// ── EXE/MEM Register ──────────────────────────────────────
EXE_MEM_reg EXE_MEM (
    .clk                  (clk),
    .reset                (reset),
    .flush                (EX_MEM_Flush),
    .EXE_Branch_in        (EXE_Branch),
    .EXE_MemRead_in       (EXE_MemRead),
    .EXE_MemtoReg_in      (EXE_MemtoReg),
    .EXE_MemWrite_in      (EXE_MemWrite),
    .EXE_RegWrite_in      (EXE_RegWrite),
    .EXE_Jump_in          (EXE_Jump),
    .EXE_adder_result_in  (branch_target),
    .EXE_ALU_result_in    (alu_result),
    .EXE_read_data_in     (EXE_Rd),
    .EXE_funct3_in        (EXE_funct3),
    .EXE_write_data_in    (alu_in2_fwd),  // forwarded store data
    .EXE_zero_in          (zero),
    .MEM_Branch_out       (MEM_Branch),
    .MEM_MemRead_out      (MEM_MemRead),
    .MEM_MemtoReg_out     (MEM_MemtoReg),
    .MEM_MemWrite_out     (MEM_MemWrite),
    .MEM_RegWrite_out     (MEM_RegWrite),
    .MEM_Jump_out         (MEM_Jump),
    .MEM_adder_result_out (MEM_adder_result),
    .MEM_ALU_result_out   (MEM_ALU_result),
    .MEM_read_data_out    (MEM_Rd),
    .MEM_funct3_out       (MEM_funct3),
    .MEM_write_data_out   (MEM_write_data),
    .MEM_zero_out         (MEM_zero)
);

// ── MEM Stage ─────────────────────────────────────────────
data_memory DMEM (
    .clk        (clk),
    .reset      (reset),
    .mem_write  (MEM_MemWrite),
    .mem_read   (MEM_MemRead),
    .funct3     (MEM_funct3),
    .Address    (MEM_ALU_result),
    .write_data (MEM_write_data),
    .read_data  (read_data_mem)
);
```

```
AND_gate AND_G (
    .zero   (MEM_zero),
    .branch (MEM_Branch),
    .out    (and_out)
);

OR_gate OR_G (
    .jump            (MEM_Jump),
    .AND_gate_output (and_out),
    .OR_gate_result  (or_out)
);

assign pc_src = or_out;

// PC MUX (2-input) — selects PC+4 or branch/jump target
mux PC_MUX (
    .pc_out       (pc_plus4),
    .adder_output (MEM_adder_result),
    .sel          (pc_src),
    .mux_output   (pc_in)
);

// ── MEM/WB Register ──────────────────────────────────────
MEM_WB_reg MEM_WB (
    .clk                (clk),
    .reset              (reset),
    .MEM_MemtoReg_in    (MEM_MemtoReg),
    .MEM_RegWrite_in    (MEM_RegWrite),
    .MEM_ALU_result_in  (MEM_ALU_result),
    .MEM_Rd_in          (MEM_Rd),
    .MEM_read_data_in   (read_data_mem),
    .WB_MemtoReg_out    (WB_MemtoReg),
    .WB_RegWrite_out    (WB_RegWrite),
    .WB_ALU_result_out  (WB_ALU_result),
    .WB_Rd_out          (WB_Rd),
    .WB_read_data_out   (WB_read_data)
);

endmodule
```

Test Bench

```
`timescale 1ns/1ps
module testbench;
    // Inputs
    reg clk;
    reg reset;
    // Instantiate the TOP module
    tope uut (
        .clk(clk),
        .reset(reset)
    );
    // Clock generation
    initial
    begin
        clk = 0;
        forever #5 clk = ~clk;
    end
    // Reset generation
    initial
    begin
        reset = 1;
        #10;
        reset = 0;
    end
    // Generate VCD file for GTKWave
    initial
    begin
        $dumpfile("dump.vcd");
        $dumpvars(0, testbench);
    end

    // Monitor important signals
    initial
    begin
        $monitor("TIME=%0t PC=%h INSTRUCTION=%h",
                 $time,
                 uut.pc_out,
                 uut.instruction);
    end
    // Stop simulation
    initial
    begin
        #300;
        $finish;
    end
endmodule
```

3. Verification

3.1. Verifying Instructions

First of all, we will verify whether our RV32I processor is running all the 47 base line instructions or not.

3.1.1. R-Type

Assume we have this instruction:

```
add x3, x6, x9
```

In this instruction we the values of registers x6 and x9 are being added and than stored in the register x3 where:

x6 = 6

x9 = 9

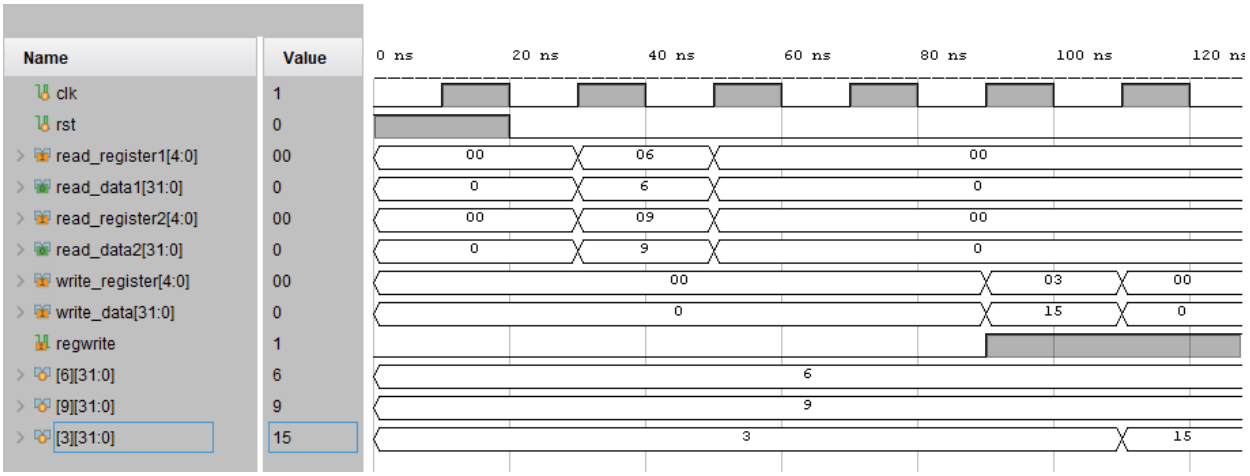


Figure 7: Verifying R-Type Instruction

As you can see the read_registers is reading from register 06 and register 09 and after adding the correct value = 15, is coming to write_data and the value of register 03 is changed from 3 to 15.

3.1.2. I-Type

For the I-Type instruction, we will run this:

```
ORI x5, x9, 3
```

In this instruction the value of register x9 is 9 and 3 is an immediate value. After performing the OR operation, the result will be stored in register x5.

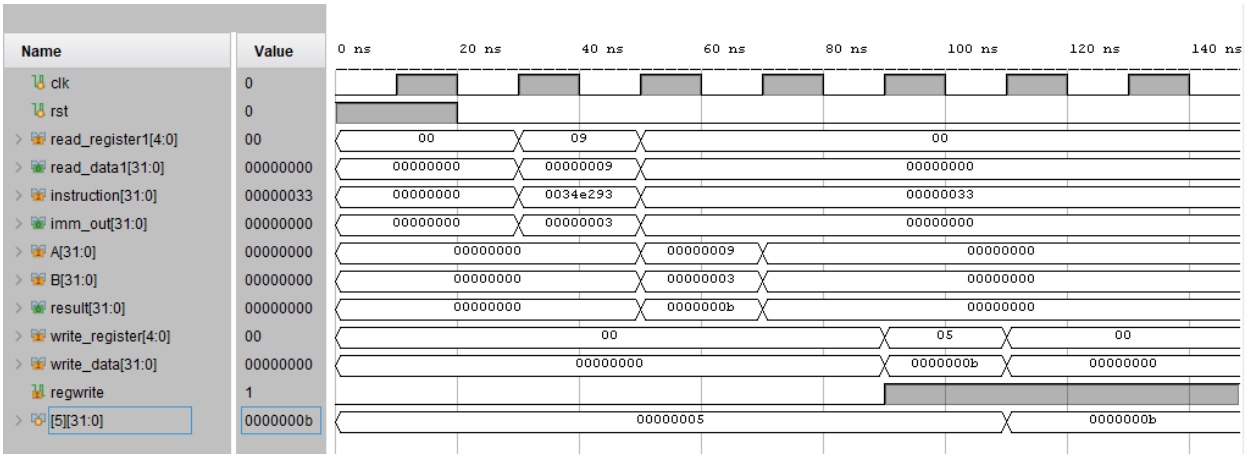


Figure 8: Verifying I-Type Instruction

As you can see the read register 1 is correctly reading the data of register x9 and the immediate generator is correctly extracting the immediate value from the instruction. Then these values go to ALU and the OR of 9 and 3 is performed and the result is B which is being stored in register x5.

3.1.3. U-Type

For the U-type instruction we will take the example of lui instruction:

```
lui x4, 0x12345
```

In this instruction we the this value should be stored inside the x4 register after left shift.

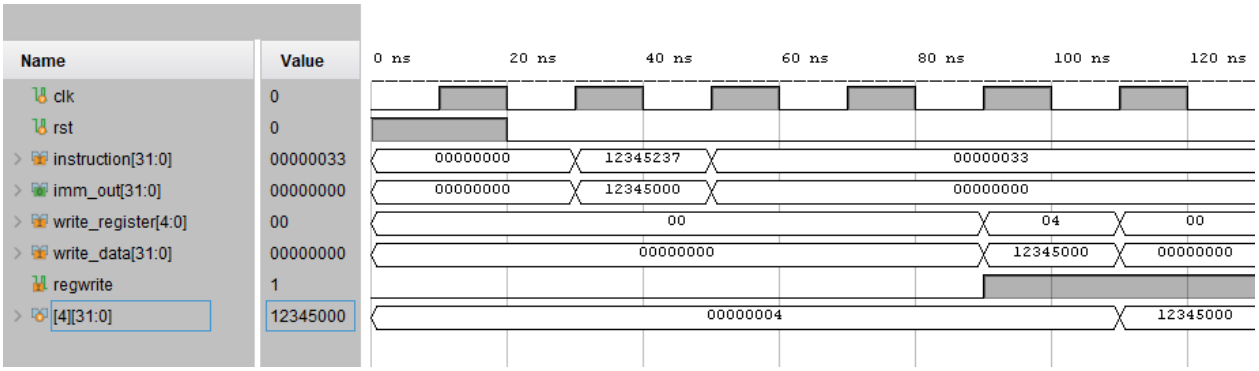


Figure 9: Verifying U-Type Instruction

In the above image, we can see that the immediate generator correctly extracted the immediate value and that value is being stored in the register 4 after left shift.

3.1.4. S-Type

For the S-Type instruction, we will take the following example:

```
sw x3, 0(x5)
```

In this instruction we want to store the value of register x3 to the 5th memory location.

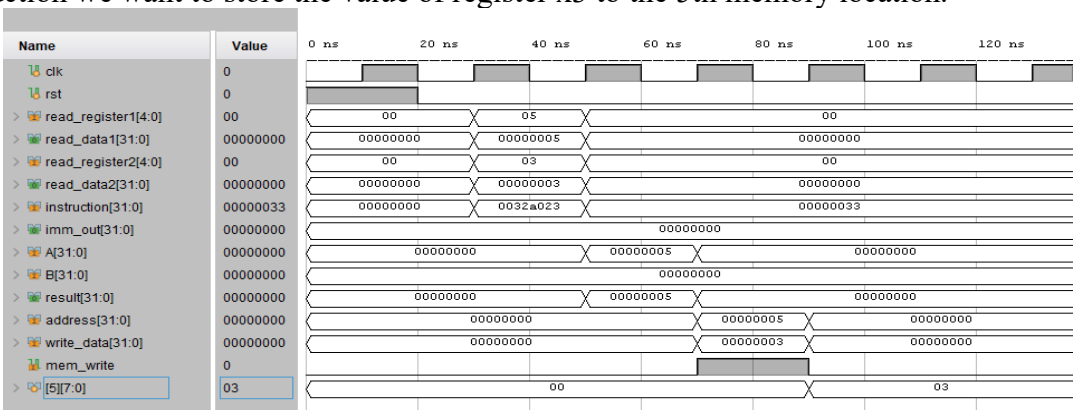


Figure 100: Verifying S-Type Instruction

It is clear that the correct values of registers are being extracted. Also the immediate generator is also extracting the correct values of immediate. The ALU is calculating the memory address where we must store the data by adding the value of register 5 (x5 = 5) with the immediate value which is 0. So, the value of register x3 which is 3 is now being stored at the 5th memory location.

3.1.5. B-Type

For this let's take the following instruction:

```
BNE x3, x5, 4
```

In this instruction the processor will check whether the values of x3 and x5 are equal or not. If not equal than branch to the address 4.

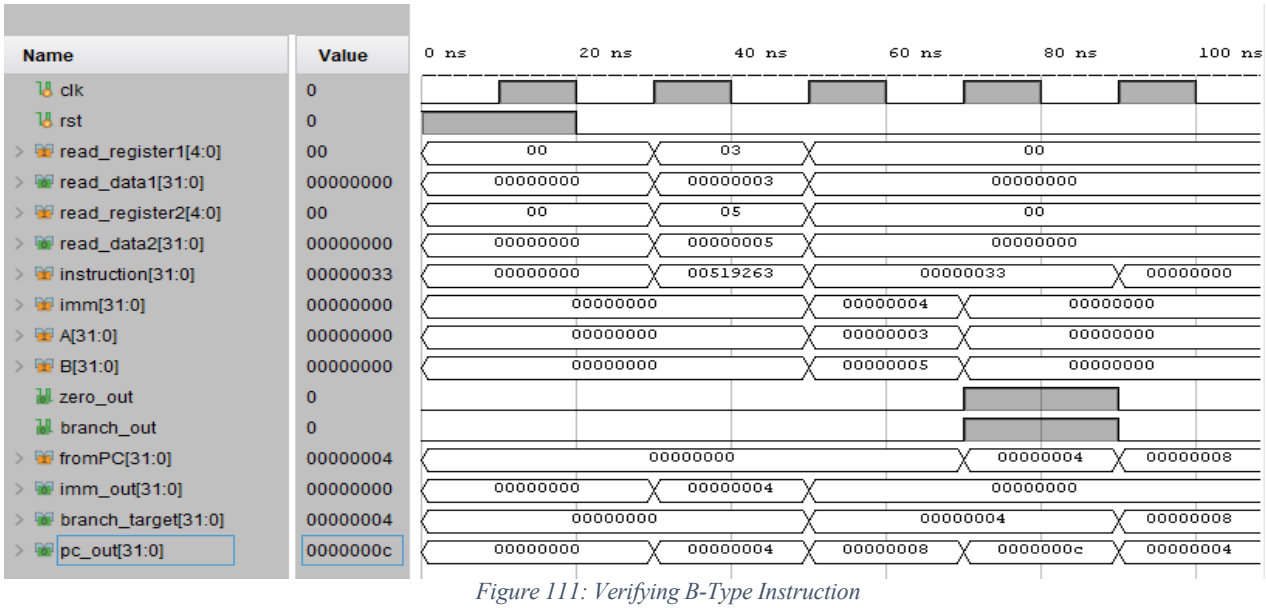


Figure 111: Verifying B-Type Instruction

We can observe that the value of register x3 is 3 and x5 is 5. ALU will check since they are not equal the zero flag will be 1 and the branch control from the control unit will be 1 so it means the

branch address ALU will calculate the addition of current PC value which is zero and add it to the immediate value which give the result = 4. So, the PC will branch to the address 4 from c.

3.1.6. J-Type

For the verification of J-Type instruction we will take the following example:

```
JAL x3, 16
```

As you can see in the below image, it is correctly following the instruction. It is jumping to PC+16 value and storing the next PC value to register x.

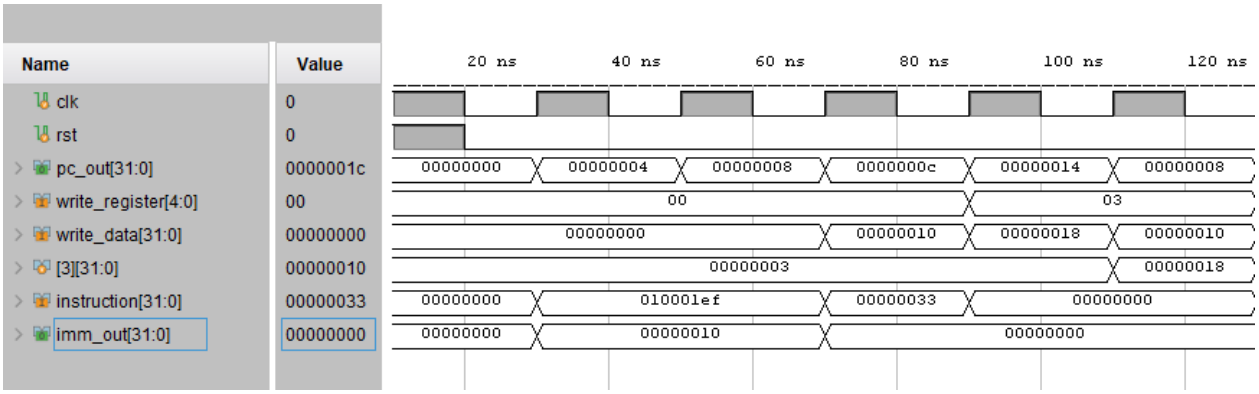


Figure 12: Verifying J-Type Instruction

Hence our RV32I Processor can run all 47 instructions.

3.2. Verifying Pipelining

We verified the behaviour of pipelining by running a test C program and it showed us that all the pipelined registers are working correctly and all the blocks of RV32I are getting the desired values at correct timing.

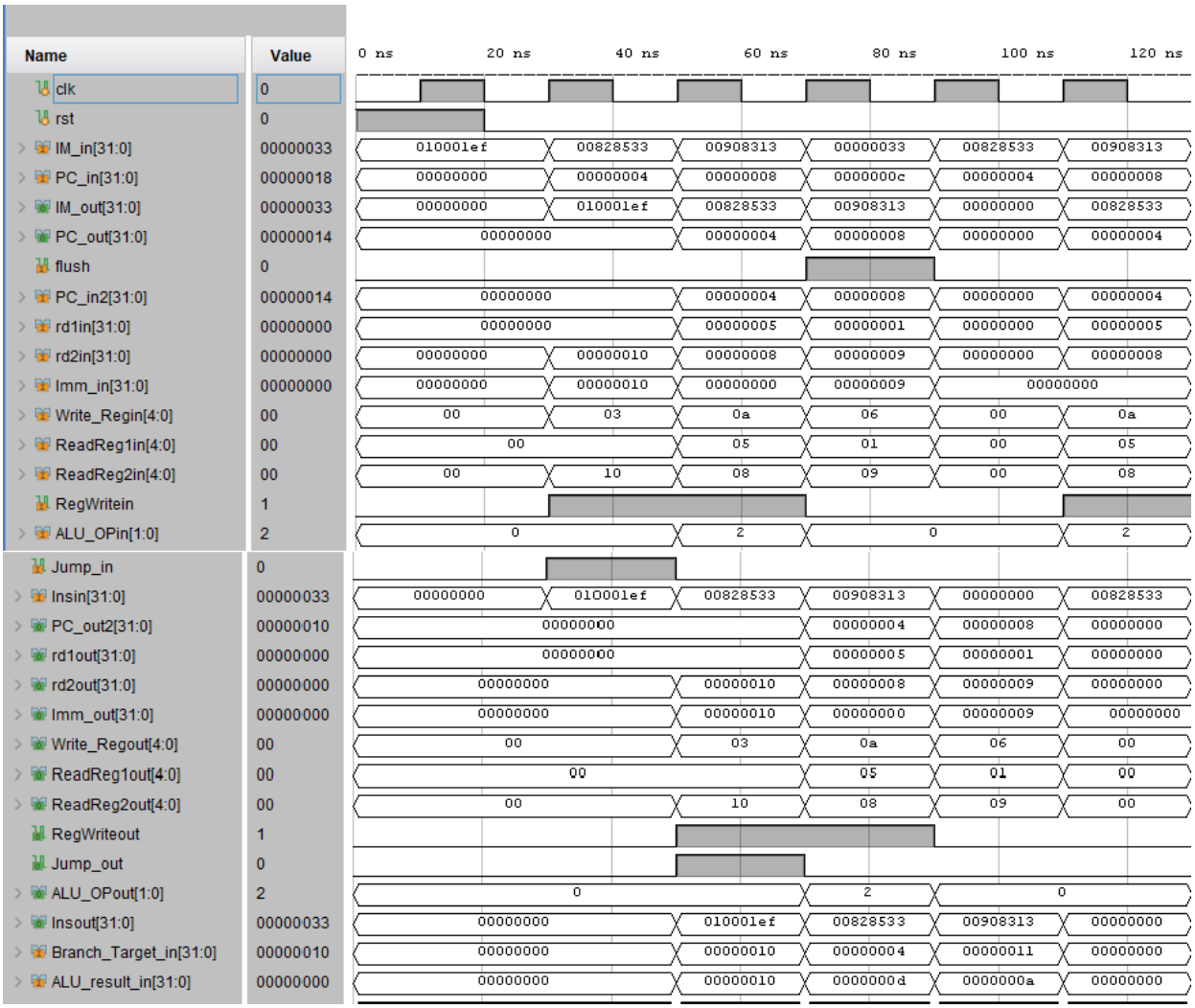


Figure 13: Verifying Pipeline Registers

3.3. Verifying Hazard Controls

Now let's verify our processor's response to the hazards.

3.3.1. Data Hazard

To check the data hazard control, we will run these instructions to the processor:

```
add x3, x5, x2
add x9, x3, x8
```

After running this using the C code, we get this output:

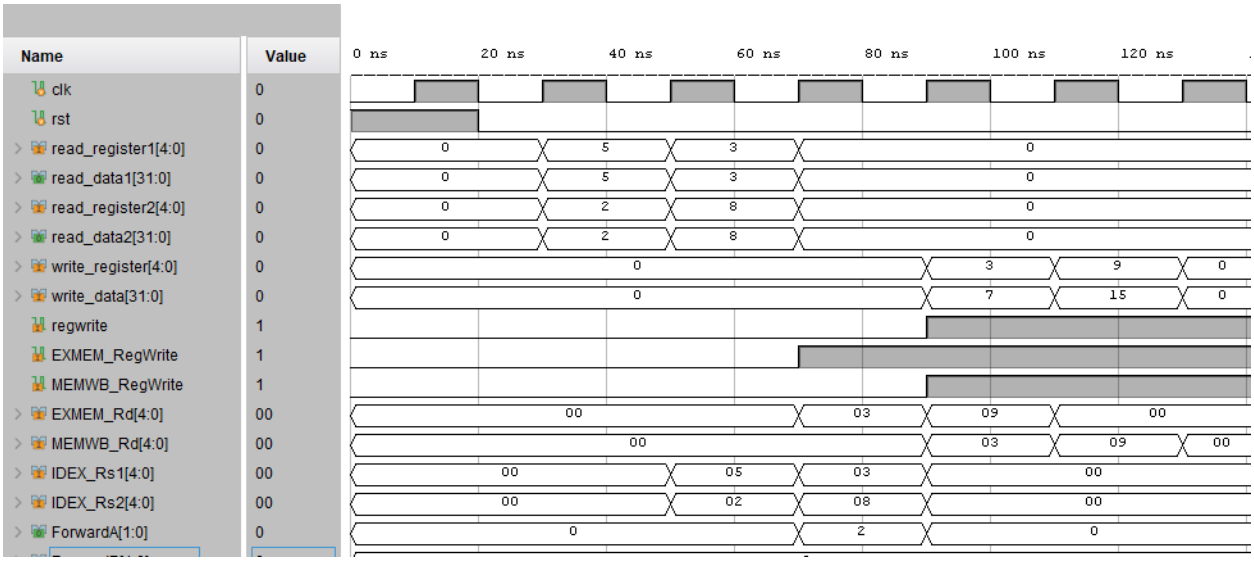


Figure 14: Data Hazard Verification

By looking at the 'write_register' and 'write_data' signal, we can observe that 7 is being stored in x3 and than this value is being used by the next instruction and after addition with 8, the result is being stored in the x9 register correctly. This is happening because the forwarding unit is correctly passing the values and controlling the signals.

3.3.2. Control Hazard

For the control hazard verification, we will use the following C program:

```
if (x1 != x2)
 { x3 = 1; }
x3 = 2;
```

In Assembly the above code becomes:

```
bne  x1, x2, LABEL
addi x3, x0, 1
LABEL:
addi x3, x0, 2
```

By looking at the output given below, we can observe that when it verifies the branch decision was correct, the 'ControlFlush' signal is being high and it will cancel the '`addi x3, x0, 1`' instruction.

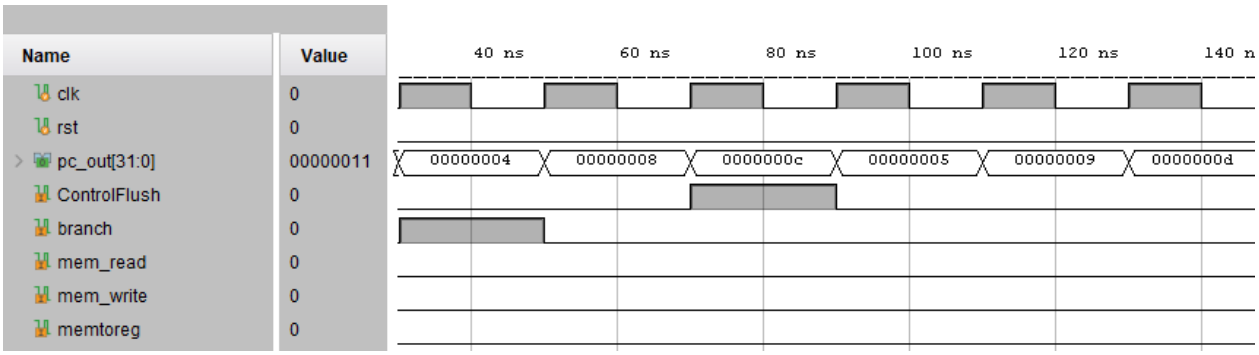
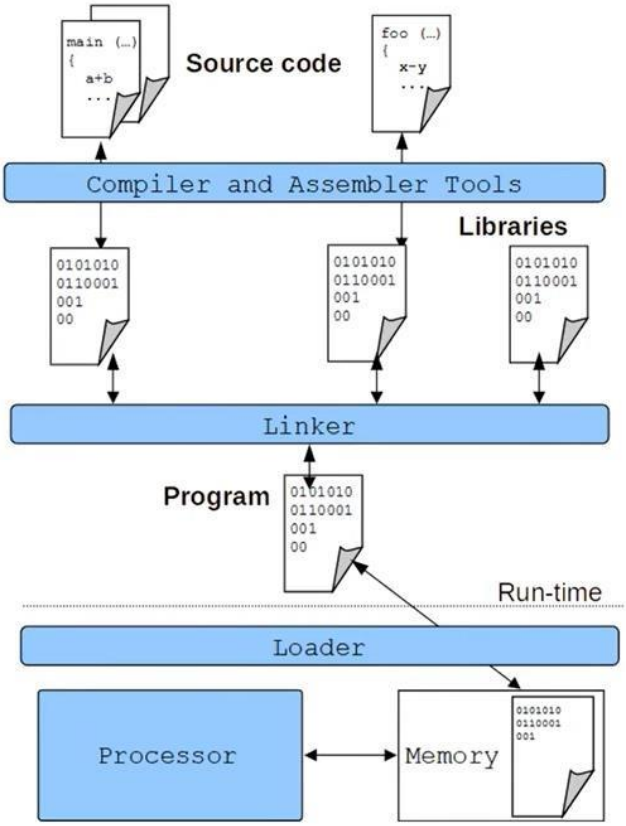


Figure 15: Control Hazard Verification

4. GCC Compatibility

We also made out processor GCC compatible means we can test the processor by using C code. Here is the flow of this:

The process starts with the source code written in C, which is easy for humans to understand but cannot be executed directly by the processor. This source code is first given to the compiler, which translates the C statements into RISC-V assembly instructions while performing necessary checks and optimizations. The assembler then converts these assembly instructions into binary machine code, which is usually represented in hexadecimal form for convenience. At the same time, any required libraries are also compiled and assembled into machine code. The linker combines all the generated object files and library code into a single executable program, resolving function calls and assigning correct memory addresses. The output of the linker is the final program file containing RV32I machine instructions in hex format. During run time, the loader places this program into memory and initializes the execution environment. Finally, the RV32I processor fetches the instructions from memory, decodes them according to the RISC-V instruction set, and executes them, producing the required program output.



5. Conclusion

In this project, we designed and implemented a 32-bit, 5-stage pipelined RISC-V processor that correctly follows the RV32I instruction set. The processor includes all five pipeline stages and supports all required instruction types. We handled data hazards using forwarding and stalling techniques, and control hazards were managed using proper pipeline control methods. The processor was tested using different programs, including GCC-generated RISC-V code. Simulation results showed correct instruction execution, proper pipeline behavior, and accurate handling of hazards. Overall, this project successfully demonstrates a working and reliable RV32I pipelined processor design.